MODUL 07 · ROUTING & NAVIGASI

Navigasi & Routing Multi-Halaman

Menguasai prinsip tumpukan navigasi (Navigation Stack), berpindah halaman dengan Navigator.push/pop, mengirim data lewat constructor dan menerima hasil balik, pushReplacement & pushAndRemoveUntil untuk alur login/logout, Named Routes dengan onGenerateRoute, NavigationBar + IndexedStack, Drawer, TabBar, dialog sebagai route, go_router dengan redirect, dan PopScope untuk konfirmasi keluar.

Durasi Pertemuan 7 (150 Menit) + Belajar Mandiri

Prasyarat Modul 06: Scrollable Views

Capaian Pembelajaran

- Memahami konsep tumpukan layar (LIFO: Last-In First-Out) pada Navigator.
- Menggunakan MaterialPageRoute beserta opsi fullscreenDialog dan transisi kustom.
- Mengirim data argumen ke halaman baru dan menerima data balik (return result).
- Memilih push, pushReplacement, pushAndRemoveUntil, dan popUntil sesuai alur login/logout.
- Menerapkan Named Routes dengan onGenerateRoute yang aman dari crash.
- Membangun navigasi multi-tab dengan NavigationBar + IndexedStack, Drawer, dan TabBar.
- Memperlakukan dialog dan bottom sheet sebagai route yang mengembalikan nilai.
- Memahami declarative routing dengan go_router (path parameter, go vs push, redirect).
- Mencegat tombol back dengan PopScope untuk konfirmasi keluar.

Daftar Isi Modul

1. Konsep Tumpukan Navigasi: push & pop
2. MaterialPageRoute & Transisi Halaman
3. Mengirim Data lewat Constructor
4. Menerima Hasil Balik dengan await push & pop
5. pushReplacement & pushAndRemoveUntil (Login/Logout)
6. Named Routes & onGenerateRoute
7. NavigationBar Material 3 & IndexedStack
8. Menu Samping dengan Drawer
9. TabBar & TabBarView
10. Dialog & Bottom Sheet sebagai Route
11. Pengenalan go_router
12. PopScope: Konfirmasi Sebelum Kembali
13. Studi Kasus: Portal Akademik Multi-Halaman
14. Tugas Praktikum
15. Kuis Pemahaman & Kunci Jawaban

1. Konsep Tumpukan Navigasi: push & pop

Navigasi di Flutter bekerja seperti **Tumpukan Piring (Stack - LIFO)**. Saat pengguna membuka halaman baru, halaman tersebut diletakkan di atas tumpukan (`push`). Saat tombol kembali ditekan, halaman paling atas diambil dan disingkirkan (`pop`).

Yang mengatur tumpukan ini adalah widget `Navigator`. Kita tidak perlu membuatnya sendiri: `MaterialApp` sudah menyediakan satu `Navigator` untuk seluruh aplikasi. Setiap "halaman" di dalam tumpukan disebut **route** (rute).

Operasi	Metode Flutter	Efek pada Tumpukan
Masuk Halaman Baru	`Navigator.push(context, route)`	Menambahkan halaman baru di puncak stack. Layar sebelumnya masih hidup di bawahnya.
Kembali	`Navigator.pop(context, [result])`	Menghapus halaman paling atas. Mengembalikan kontrol ke layar di bawahnya.
Ganti Halaman (Replace)	`Navigator.pushReplacement(context, route)`	Menghancurkan halaman saat ini dan menggantinya dengan yang baru (Contoh: dari Layar Login ke Layar Utama).
Bersihkan Seluruh Stack	`Navigator.pushAndRemoveUntil(...)`	Menghapus seluruh riwayat layar masa lalu (Contoh: Logout kembali ke Login).

Ilustrasi tumpukan

```
Awal aplikasi        Setelah push(Tentang)     Setelah pop()
+-----------+        +-----------+             +-----------+
|  Beranda  |        |  Tentang  |  <- atas    |  Beranda  |
+-----------+        +-----------+             +-----------+
                     |  Beranda  |
                     +-----------+
```

Contoh kecil: push dan pop

Dua baris ini adalah inti navigasi. Baris pertama dipanggil di halaman asal, baris kedua di halaman tujuan.

```dart
// MASUK ke halaman baru (push = taruh di atas tumpukan)
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const HalamanB()),
```

```
);

// KEMBALI ke halaman sebelumnya (pop = ambil halaman teratas)
Navigator.pop(context);
```

- `context` dipakai untuk mencari `Navigator` terdekat di atas widget ini.
- `MaterialPageRoute` membungkus halaman tujuan dan memberi animasi transisi standar (dibahas di section berikutnya).
- `Navigator.pop(context)` tidak perlu tahu halaman mana yang ada di bawahnya. Ia cukup membuang halaman teratas.

Contoh lengkap: Beranda dan Tentang

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const BerandaScreen(),
    );
  }
}

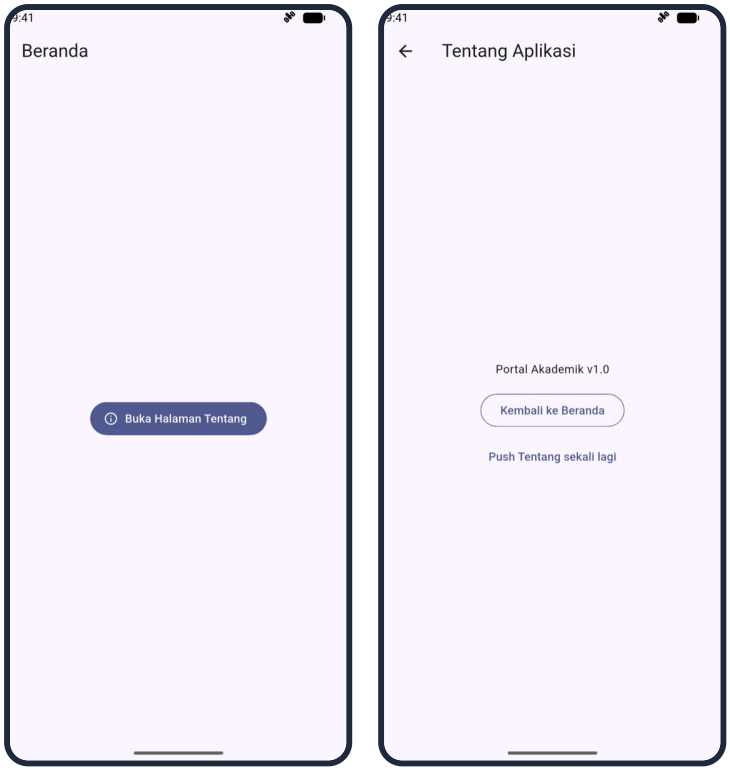
// --- HALAMAN 1 (paling bawah di tumpukan) ---
class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Beranda')),
      body: Center(
        child: FilledButton.icon(
          icon: const Icon(Icons.info_outline),
          label: const Text('Buka Halaman Tentang'),
          onPressed: () {
            // push: TentangScreen diletakkan DI ATAS BerandaScreen
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const TentangScreen()),
            );
          },
        ),
```

```
      ),
    );
  }
}

// --- HALAMAN 2 ---
class TentangScreen extends StatelessWidget {
  const TentangScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // AppBar otomatis menampilkan tombol panah kembali (←)
      // karena ada halaman lain di bawah tumpukan.
      appBar: AppBar(title: const Text('Tentang Aplikasi')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('Portal Akademik v1.0'),
            const SizedBox(height: 16),
            OutlinedButton(
              // pop: buang halaman teratas, kembali ke Beranda
              onPressed: () => Navigator.pop(context),
              child: const Text('Kembali ke Beranda'),
            ),
            const SizedBox(height: 8),
            TextButton(
              // push lagi: tumpukan menjadi Beranda -> Tentang -> Tentang
              onPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (context) => const TentangScreen()),
                );
              },
              child: const Text('Push Tentang sekali lagi'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Halaman pertama: Beranda

Setelah push: Tentang di atas Beranda (ada panah kembali)

Penjelasan:

- `home: const BerandaScreen()` menjadi route pertama (paling bawah) di tumpukan.
- Di `TentangScreen`, `AppBar` otomatis menampilkan tombol panah kembali. Tombol itu memanggil `Navigator.maybePop` untuk kita.
- Tombol "Push Tentang sekali lagi" sengaja dibuat untuk membuktikan bahwa halaman yang sama boleh ada dua kali di tumpukan. Tekan back dua kali untuk kembali ke Beranda.
- Tombol back fisik/gestur di Android juga melakukan `pop`.

⚠️ **Kesalahan umum**

- **Memanggil `pop` di halaman pertama.** Jika tumpukan tinggal satu route, `Navigator.pop` membuat layar hitam (tidak ada halaman tersisa). Gunakan `Navigator.canPop(context)` untuk mengecek, atau `Navigator.maybePop(context)` yang aman.
- **Memanggil `Navigator.push` langsung di dalam `build()`.** Navigasi harus dipicu oleh aksi (tombol, timer, hasil login), bukan setiap kali widget di-render.
- **Memakai `context` milik `MyApp` (di atas `MaterialApp`).** Context tersebut belum punya `Navigator`, sehingga muncul error *"Navigator operation requested with a context that does not include a Navigator"*. Lakukan navigasi dari widget halaman (di bawah `MaterialApp`).

2. MaterialPageRoute & Transisi Halaman

`MaterialPageRoute` adalah "bungkus" untuk halaman tujuan. Ia menentukan *bagaimana* halaman muncul: animasi transisi, apakah berupa dialog layar penuh, dan nama rutenya.

Parameter penting

```
MaterialPageRoute(
  builder: (context) => const FormScreen(), // WAJIB: membangun halaman tujuan
  settings: const RouteSettings(name: '/form'), // opsional: nama rute (untuk debug/analytics)
  fullscreenDialog: true, // opsional: animasi dari bawah + tombol X
  maintainState: true, // default true: halaman tetap hidup saat tertutup halaman lain
);
```

Parameter	Fungsi
`builder`	Fungsi yang mengembalikan widget halaman. Dipanggil saat route dibangun.
`settings`	Berisi `name` dan `arguments`. Berguna untuk named routes dan debugging.
`fullscreenDialog`	Jika `true`, halaman masuk dari bawah dan tombol back menjadi ikon X. Cocok untuk form "Tambah ...".
`maintainState`	Jika `true` (default), halaman di bawah tetap disimpan di memori sehingga isian form dan posisi scroll tidak hilang.

Contoh lengkap: tiga gaya transisi

Contoh berikut membandingkan rute biasa, `fullscreenDialog`, dan `PageRouteBuilder` untuk animasi kustom (fade).

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const PengumumanScreen(),
```

```
    );
  }
}

class PengumumanScreen extends StatelessWidget {
  const PengumumanScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pengumuman')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // 1. Rute biasa: animasi geser standar platform
          FilledButton(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  settings: const RouteSettings(name: '/detail'),
                  builder: (context) => const DetailPengumumanScreen(),
                ),
              );
            },
            child: const Text('Rute biasa (geser)'),
          ),
          const SizedBox(height: 12),
          // 2. fullscreenDialog: muncul dari bawah, tombol kembali berubah jadi X
          FilledButton.tonal(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  fullscreenDialog: true,
                  builder: (context) => const TulisPengumumanScreen(),
                ),
              );
            },
            child: const Text('fullscreenDialog (dari bawah)'),
          ),
          const SizedBox(height: 12),
          // 3. PageRouteBuilder: animasi kustom (fade)
          OutlinedButton(
            onPressed: () {
              Navigator.push(
                context,
                PageRouteBuilder(
                  transitionDuration: const Duration(milliseconds: 400),
                  pageBuilder: (context, animation, secondaryAnimation) =>
                      const DetailPengumumanScreen(),
                  transitionsBuilder:
                      (context, animation, secondaryAnimation, child) {
                    return FadeTransition(opacity: animation, child: child);
                  },
                ),
              );
            },
            child: const Text('PageRouteBuilder (fade)'),
```

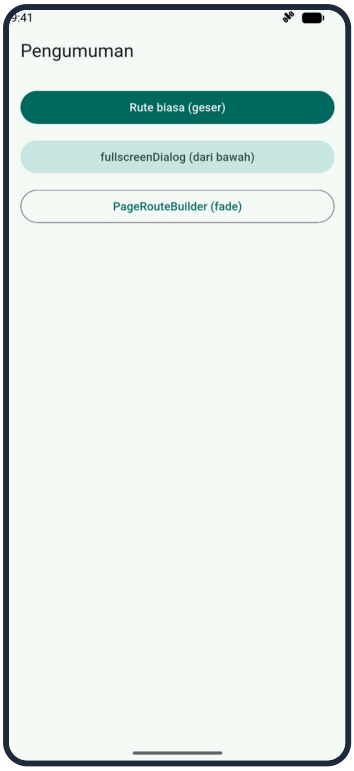
```
          ),
        ],
      ),
    );
  }
}

class DetailPengumumanScreen extends StatelessWidget {
  const DetailPengumumanScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // ModalRoute.of(context) memberi akses ke rute yang sedang aktif
    final namaRute = ModalRoute.of(context)?.settings.name ?? '(tanpa nama)';
    return Scaffold(
      appBar: AppBar(title: const Text('Detail Pengumuman')),
      body: Center(child: Text('Nama rute: $namaRute')),
    );
  }
}

class TulisPengumumanScreen extends StatelessWidget {
  const TulisPengumumanScreen({super.key});

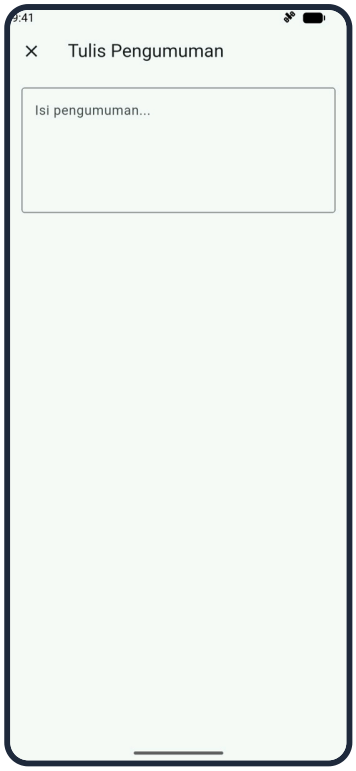
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tulis Pengumuman')),
      body: const Padding(
        padding: EdgeInsets.all(16),
        child: TextField(
          maxLines: 5,
          decoration: InputDecoration(
            border: OutlineInputBorder(),
            hintText: 'Isi pengumuman...',
          ),
        ),
      ),
    );
  }
}
```



Tiga tombol untuk tiga jenis rute



Rute biasa: nama rute terbaca dari settings



fullscreenDialog: tombol kembali berupa ikon X

Penjelasan:

- `MaterialPageRoute` otomatis menyesuaikan animasi dengan platform (Android: zoom/geser, iOS: geser dari kanan).
- `ModalRoute.of(context)?.settings.name` membaca nama rute dari dalam halaman tujuan. Rute ketiga tidak diberi `settings`, jadi namanya `null`.
- `PageRouteBuilder` memberi kendali penuh: `pageBuilder` membangun halaman, `transitionsBuilder` membungkusnya dengan animasi. Nilai `animation` bergerak dari 0.0 ke 1.0.

> 💡 **Tips**
>
> Untuk aplikasi kuliah, `MaterialPageRoute` sudah cukup di 95% kasus. Pakai `PageRouteBuilder` hanya jika desain memang meminta animasi khusus.

⚠️ **Kesalahan umum**

- Menulis `builder: (context) => HalamanB` (tanpa tanda kurung). Yang benar adalah memanggil constructor: `HalamanB()`.
- Mengira `fullscreenDialog` sama dengan `showDialog`. `fullscreenDialog` tetap halaman penuh, hanya gaya animasinya yang berbeda.

3. Mengirim Data lewat Constructor

Cara paling sederhana dan paling aman untuk mengirim data ke halaman baru adalah lewat **constructor**. Halaman tujuan hanyalah sebuah class Dart, jadi data cukup dikirim sebagai argumen.

Pola dasarnya

```
// Halaman ASAL: kirim data lewat argumen constructor
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const ProfilScreen(nama: 'Sinta', angkatan: 2023),
  ),
);


// Halaman TUJUAN: deklarasikan field final + parameter required
class ProfilScreen extends StatelessWidget {
  final String nama;
  final int angkatan;

  const ProfilScreen({super.key, required this.nama, required this.angkatan});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(nama)),
      body: Center(child: Text('Angkatan $angkatan')),
    );
  }
}
```

- Field di halaman tujuan dibuat `final` karena data kiriman tidak berubah selama halaman tampil.
- Parameter `required` membuat compiler protes jika kita lupa mengirim datanya. Kesalahan ketahuan sebelum aplikasi dijalankan.
- Jika halaman tujuan berupa `StatefulWidget`, data dibaca di class State dengan `widget.nama`.

Contoh lengkap: daftar mata kuliah ke halaman detail

Daripada mengirim `kode`, `nama`, `sks` satu per satu, kirim satu objek model utuh.

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

```dart
// Model data sederhana (idealnya di lib/models/mata_kuliah.dart)
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;
  final String dosen;

  const MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.dosen,
  });
}

const daftarMatkul = [
  MataKuliah(kode: 'IF201', nama: 'Pemrograman Mobile', sks: 3, dosen: 'Dr. Rina'),
  MataKuliah(kode: 'IF202', nama: 'Basis Data', sks: 3, dosen: 'Budi, M.Kom.'),
  MataKuliah(kode: 'IF203', nama: 'Jaringan Komputer', sks: 2, dosen: 'Sari, M.T.'),
  MataKuliah(kode: 'IF204', nama: 'Kecerdasan Buatan', sks: 3, dosen: 'Dr. Agus'),
];

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarMatkulScreen(),
    );
  }
}

class DaftarMatkulScreen extends StatelessWidget {
  const DaftarMatkulScreen({super.key});

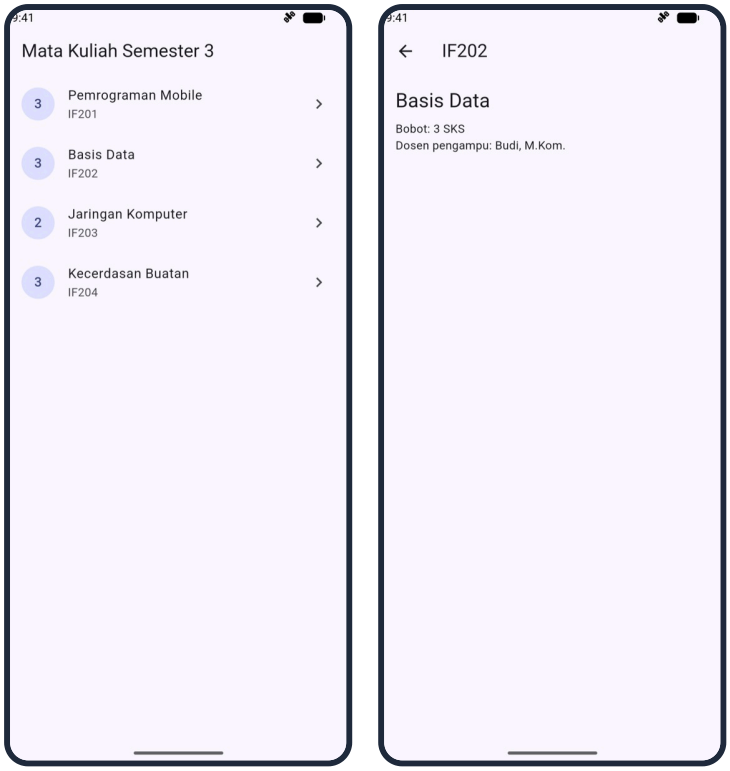
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah Semester 3')),
      body: ListView.builder(
        itemCount: daftarMatkul.length,
        itemBuilder: (context, index) {
          final mk = daftarMatkul[index];
          return ListTile(
            leading: CircleAvatar(child: Text('${mk.sks}')),
            title: Text(mk.nama),
            subtitle: Text(mk.kode),
            trailing: const Icon(Icons.chevron_right),
            onTap: () {
              // Kirim SATU objek utuh, bukan field satu per satu
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => DetailMatkulScreen(matkul: mk),
                ),
              );
```

```
            },
          );
        },
      ),
    );
  }
}

class DetailMatkulScreen extends StatelessWidget {
  // Data yang diterima dari halaman sebelumnya
  final MataKuliah matkul;

  const DetailMatkulScreen({super.key, required this.matkul});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(matkul.kode)),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(matkul.nama, style: Theme.of(context).textTheme.headlineSmall),
            const SizedBox(height: 8),
            Text('Bobot: ${matkul.sks} SKS'),
            Text('Dosen pengampu: ${matkul.dosen}'),
          ],
        ),
      ),
    );
  }
}
```



Daftar mata kuliah

Detail menerima objek MataKuliah lewat constructor

Penjelasan:

- Class `MataKuliah` adalah model data. Di proyek nyata letakkan di `lib/models/`.
- Di `onTap`, objek `mk` dikirim ke `DetailMatkulScreen(matkul: mk)`.
- Halaman detail tidak perlu tahu dari mana datanya berasal. Ia hanya menampilkan `matkul` yang diterima. Pemisahan ini membuat halaman mudah dipakai ulang dan diuji.

> ℹ️ **Constructor vs arguments**
>
> Selain constructor, Flutter juga punya `RouteSettings.arguments` (dipakai di named routes, section 6). Constructor lebih disarankan karena tipe datanya diperiksa compiler, sedangkan `arguments` bertipe `Object?` dan harus dicek manual.

⚠️ **Kesalahan umum**

- Menyimpan data kiriman di variabel global agar "bisa diakses dari halaman lain". Ini membuat data sulit dilacak. Kirim lewat constructor, atau pakai controller GetX (Modul 08) untuk data yang dipakai banyak halaman.
- Lupa menulis `widget.` saat membaca field di `State`: tulis `widget.matkul.nama`, bukan `matkul.nama`.

4. Menerima Hasil Balik dengan await push & pop

Navigasi bisa dua arah. `Navigator.push` mengembalikan sebuah `Future` yang selesai ketika halaman tujuan ditutup. Nilai yang dikirim lewat `Navigator.pop(context, nilai)` akan menjadi hasil `Future` tersebut.

Pola dasarnya

```
// Halaman ASAL: tunggu hasil dengan await
onPressed: () async {
  final String? kelas = await Navigator.push<String>(
    context,
    MaterialPageRoute(builder: (context) => const PilihKelasScreen()),
  );
  if (kelas == null) return; // pengguna menekan tombol back
  debugPrint('Kelas dipilih: $kelas');
},


// Halaman TUJUAN: tutup sambil membawa nilai
onTap: () => Navigator.pop(context, 'Kelas A'),
```

- `push<String>` memberi tahu Dart bahwa hasilnya bertipe `String?`. Tanda tanya muncul karena pengguna bisa menekan back tanpa memilih apa pun, sehingga hasilnya `null`.
- `await` membuat kode "menunggu" sampai halaman tujuan di-`pop`. Karena itu fungsi `onPressed` harus `async`.

Contoh lengkap 1: verifikasi data mahasiswa

Mari pelajari cara mengirim data objek Mahasiswa ke halaman detail, dan menerima konfirmasi status kembali:

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DaftarMahasiswaScreen(),
    );
  }
}
```

```dart
// --- HALAMAN 1: DAFTAR MAHASISWA ---
class DaftarMahasiswaScreen extends StatelessWidget {
  const DaftarMahasiswaScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Daftar Mahasiswa")),
      body: Center(
        child: ElevatedButton(
          onPressed: () async {
            // 1. Mengirim data ke DetailMahasiswaScreen dan MENUNGGU hasil balik (await)
            //    <String> = tipe data yang diharapkan dikirim balik oleh Navigator.pop
            final hasilBalik = await Navigator.push<String>(
              context,
              MaterialPageRoute(
                builder: (context) => const DetailMahasiswaScreen(
                  nama: "Fikri Ramadhan",
                  nim: "21102045",
                ),
              ),
            );

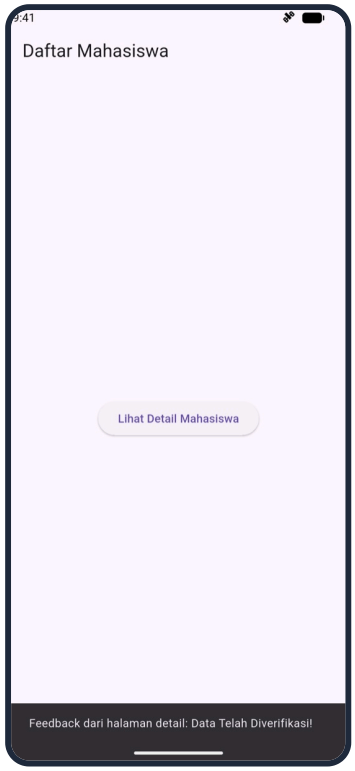
            // 2. Menerima data yang dikirim balik dari Halaman 2.
            //    Cek context.mounted karena ada jeda (await) sebelum memakai context lagi.
            if (hasilBalik != null && context.mounted) {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text("Feedback dari halaman detail: $hasilBalik")),
              );
            }
          },
          child: const Text("Lihat Detail Mahasiswa"),
        ),
      ),
    );
  }
}

// --- HALAMAN 2: DETAIL MAHASISWA ---
class DetailMahasiswaScreen extends StatelessWidget {
  final String nama;
  final String nim;

  const DetailMahasiswaScreen({super.key, required this.nama, required this.nim});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Profil $nama")),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text("NIM: $nim", style: const TextStyle(fontSize: 18)),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                // Menutup layar DAN mengembalikan data String ke halaman sebelumnya
```

```
                Navigator.pop(context, "Data Telah Diverifikasi!");
              },
              child: const Text("Verifikasi & Kembali"),
            ),
          ],
        ),
      ),
    );
  }
}
```



Hasil balik dari pop tampil di SnackBar

Contoh lengkap 2: form tambah catatan yang mengembalikan objek

Pola ini sangat sering dipakai: halaman daftar membuka halaman form, form mengembalikan objek baru, lalu daftar menambahkannya dengan `setState`.

lib/main.dart

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class Catatan {
  final String judul;
  final bool penting;

  const Catatan({required this.judul, required this.penting});
}
```

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
      home: const DaftarCatatanScreen(),
    );
  }
}

// Halaman daftar perlu StatefulWidget karena isinya berubah
class DaftarCatatanScreen extends StatefulWidget {
  const DaftarCatatanScreen({super.key});

  @override
  State<DaftarCatatanScreen> createState() => _DaftarCatatanScreenState();
}

class _DaftarCatatanScreenState extends State<DaftarCatatanScreen> {
  final List<Catatan> _catatan = [];

  Future<void> _bukaForm() async {
    // 1. Buka form dan TUNGGU sampai form ditutup
    final hasil = await Navigator.push<Catatan>(
      context,
      MaterialPageRoute(builder: (context) => const FormCatatanScreen()),
    );

    // 2. hasil == null berarti pengguna batal (tekan back)
    if (hasil == null) return;

    // 3. Setelah await, pastikan State masih terpasang sebelum setState/context
    if (!mounted) return;
    setState(() => _catatan.add(hasil));
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Catatan "${hasil.judul}" ditambahkan')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Kuliah')),
      body: _catatan.isEmpty
          ? const Center(child: Text('Belum ada catatan. Tekan tombol +'))
          : ListView.builder(
              itemCount: _catatan.length,
              itemBuilder: (context, index) {
                final c = _catatan[index];
                return ListTile(
                  leading: Icon(
                    c.penting ? Icons.star : Icons.notes,
                    color: c.penting ? Colors.amber : null,
                  ),
                  title: Text(c.judul),
```

```
                );
              },
            ),
      floatingActionButton: FloatingActionButton(
        tooltip: 'Tambah catatan',
        onPressed: _bukaForm,
        child: const Icon(Icons.add),
      ),
    );
  }
}

class FormCatatanScreen extends StatefulWidget {
  const FormCatatanScreen({super.key});

  @override
  State<FormCatatanScreen> createState() => _FormCatatanScreenState();
}

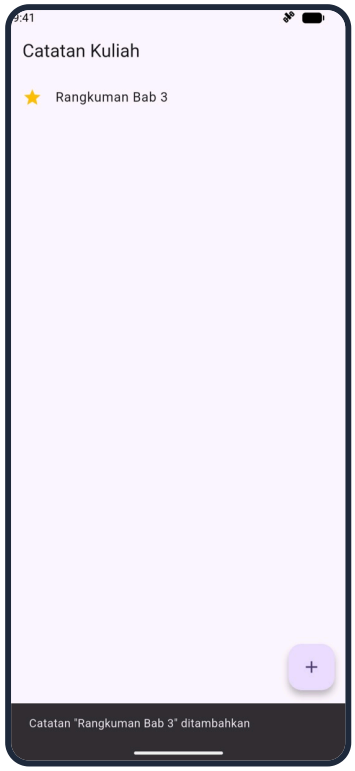
class _FormCatatanScreenState extends State<FormCatatanScreen> {
  final _judulController = TextEditingController();
  bool _penting = false;

  @override
  void dispose() {
    _judulController.dispose();
    super.dispose();
  }

  void _simpan() {
    final judul = _judulController.text.trim();
    if (judul.isEmpty) return;
    // Tutup halaman sambil MENGIRIM objek Catatan ke halaman sebelumnya
    Navigator.pop(context, Catatan(judul: judul, penting: _penting));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Baru')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _judulController,
              autofocus: true,
              decoration: const InputDecoration(
                labelText: 'Judul catatan',
                border: OutlineInputBorder(),
              ),
            ),
            SwitchListTile(
              title: const Text('Tandai penting'),
              value: _penting,
              onChanged: (v) => setState(() => _penting = v),
            ),
            const SizedBox(height: 12),
            SizedBox(
```

```
              width: double.infinity,
              child: FilledButton(onPressed: _simpan, child: const Text('Simpan')),
            ),
          ],
        ),
      ),
    );
  }
}
```



Objek Catatan dikembalikan dan ditambahkan ke daftar

Penjelasan:

- `_bukaForm()` menunggu hasil bertipe `Catatan?`. Jika `null`, berarti pengguna batal sehingga fungsi langsung berhenti.
- `if (!mounted) return;` wajib ada setelah `await`. Selama menunggu, halaman daftar bisa saja sudah ditutup. Memanggil `setState` pada State yang sudah dilepas akan menimbulkan error.
- Di form, `Navigator.pop(context, Catatan(...))` sekaligus menutup halaman dan mengirim objek.
- Di dalam `State` gunakan `mounted`; di dalam fungsi yang hanya punya `BuildContext` (seperti contoh 1) gunakan `context.mounted`.

⚠️ **Kesalahan umum**

- Lupa `await`: kode setelah `push` langsung jalan sebelum pengguna memilih apa pun, dan hasilnya selalu berupa `Future`, bukan nilai.
- Tidak menangani `null`. Pengguna selalu bisa menekan tombol back, jadi hasil *pasti* bisa `null`.
- Tipe hasil tidak cocok, misalnya `push<String>` tetapi halaman tujuan memanggil `pop(context, 123)`. Ini menimbulkan error tipe saat runtime.
- Memakai `context` setelah `await` tanpa cek `mounted`. Analyzer akan memberi peringatan *use_build_context_synchronously*.

5. pushReplacement & pushAndRemoveUntil (Login/Logout)

Tidak semua halaman boleh "dikembalikan" dengan tombol back. Setelah login, pengguna tidak seharusnya kembali ke form login. Setelah logout, pengguna tidak boleh kembali ke halaman berisi data pribadi. Untuk kasus ini kita memakai `pushReplacement` dan `pushAndRemoveUntil`.

Metode	Tumpukan sebelum	Tumpukan sesudah	Kasus
`push(Home)`	[Login]	[Login, Home]	Halaman detail biasa
`pushReplacement(Home)`	[Login]	[Home]	Splash → Login, Login → Home
`pushAndRemoveUntil(Login, (r) => false)`	[Home, Pengaturan]	[Login]	Logout
`popUntil((r) => r.isFirst)`	[Home, A, B]	[Home]	Tombol "Kembali ke Beranda"

Contoh kecil

```
// 1) Ganti halaman saat ini: [Login] menjadi [Home]
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => const HomeScreen()),
);


// 2) Push Login lalu hapus SEMUA rute di bawahnya
//    (route) => false artinya "jangan sisakan satu rute pun"
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(builder: (context) => const LoginScreen()),
  (route) => false,
);

// 3) Kembali terus sampai ke rute pertama (tanpa push baru)
Navigator.popUntil(context, (route) => route.isFirst);
```

Parameter ketiga `pushAndRemoveUntil` adalah *predicate*: fungsi yang dipanggil untuk setiap route lama, dari atas ke bawah. Route dibuang selama fungsi mengembalikan `false`. Dengan `(route) => false`, semua route lama dibuang.

Contoh lengkap: Splash → Login → Home → Pengaturan → Logout

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const SplashScreen(),
    );
  }
}

// ---------------- SPLASH ----------------
class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    _lanjutKeLogin();
  }

  Future<void> _lanjutKeLogin() async {
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return; // halaman bisa saja sudah ditutup selama menunggu
    // Splash diganti Login -> tombol back tidak akan kembali ke splash
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => const LoginScreen()),
    );
  }

  @override
  Widget build(BuildContext context) {
    return const Scaffold(
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            FlutterLogo(size: 96),
            SizedBox(height: 16),
```

```
            CircularProgressIndicator(),
          ],
        ),
      ),
    );
  }
}

// ---------------- LOGIN ----------------
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

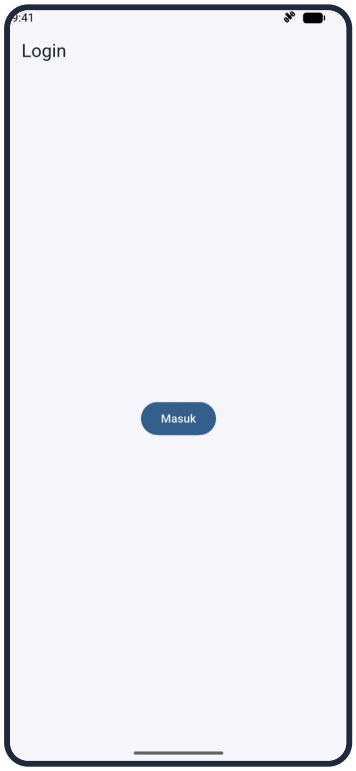
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Center(
        child: FilledButton(
          onPressed: () {
            // Login sukses: GANTI halaman login dengan Home
            Navigator.pushReplacement(
              context,
              MaterialPageRoute(builder: (context) => const HomeScreen()),
            );
          },
          child: const Text('Masuk'),
        ),
      ),
    );
  }
}

// ---------------- HOME ----------------
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

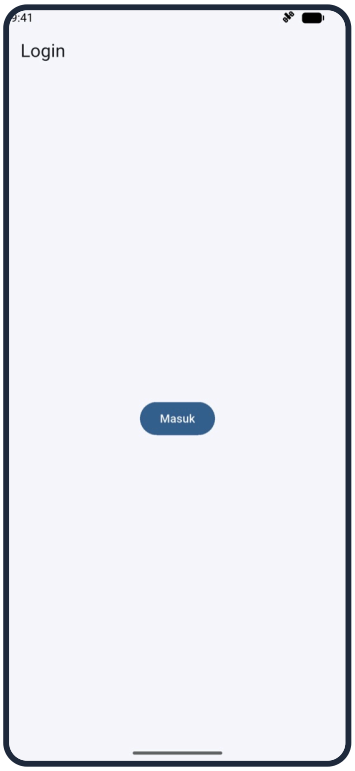
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Tidak ada panah back di sini karena Home adalah satu-satunya rute
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: FilledButton.tonal(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const PengaturanScreen()),
            );
          },
          child: const Text('Buka Pengaturan'),
        ),
      ),
    );
  }
}

// ---------------- PENGATURAN ----------------
class PengaturanScreen extends StatelessWidget {
  const PengaturanScreen({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: Center(
        child: FilledButton.icon(
          style: FilledButton.styleFrom(backgroundColor: Colors.red),
          icon: const Icon(Icons.logout),
          label: const Text('Logout'),
          onPressed: () {
            // Stack saat ini: [Home, Pengaturan]
            // Setelah ini  : [Login]  (Home & Pengaturan dibuang semua)
            Navigator.pushAndRemoveUntil(
              context,
              MaterialPageRoute(builder: (context) => const LoginScreen()),
              (route) => false,
            );
          },
        ),
      ),
    );
  }
}
```



Splash sudah diganti Login (tanpa panah kembali)



Setelah logout: tumpukan hanya [Login]

Penjelasan alur tumpukan:

1. Aplikasi mulai: `[Splash]`. Setelah 2 detik, `pushReplacement` → `[Login]`.
2. Tekan Masuk: `pushReplacement` → `[Home]`. Perhatikan AppBar Home tidak punya panah kembali.

3. Buka Pengaturan: `push` → `[Home, Pengaturan]`.
4. Logout: `pushAndRemoveUntil` → `[Login]`. Tekan back di Android, aplikasi tertutup karena tidak ada halaman lain.

> ℹ️ **Hubungan dengan Modul 13–14**
>
> Di proyek CRUD + Auth nanti, perpindahan login/home tidak lagi memakai `pushReplacement` manual, melainkan sebuah *auth gate*: widget yang memilih menampilkan Login atau Home berdasarkan status sesi di `AuthController` (GetX). Konsepnya sama: halaman login tidak boleh tersisa di tumpukan setelah login.

> ⚠️ **Kesalahan umum**
>
> - Memakai `push` biasa setelah login sehingga tombol back membawa pengguna kembali ke form login.
> - Logout dengan `pushReplacement`: hanya halaman teratas yang diganti, halaman Home masih tersisa di bawah.
> - Navigasi di `initState` setelah `Future.delayed` tanpa cek `mounted`.

6. Named Routes & onGenerateRoute

Named routes memberi setiap halaman sebuah nama seperti URL (`'/detail'`). Navigasi cukup menyebut nama, tanpa perlu meng-import class halaman tujuan di setiap file.

Cara 1: tabel `routes` (paling sederhana)

```
return MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const BerandaScreen(),
    '/tentang': (context) => const TentangScreen(),
  },
);


// Di halaman mana pun:
onPressed: () => Navigator.pushNamed(context, '/tentang'),
```

Tabel `routes` cocok untuk halaman tanpa argumen. Kelemahannya: jika halaman butuh data, kita harus membaca `ModalRoute.of(context)!.settings.arguments` di dalam halaman, dan tidak ada penanganan nama rute yang salah.

Cara 2: `onGenerateRoute` (lebih aman)

`onGenerateRoute` adalah satu fungsi pusat yang dipanggil setiap kali `pushNamed` dijalankan. Di sana kita bisa memeriksa argumen dan membuat halaman error jika rute tidak dikenal.

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class Mahasiswa {
  final String nim;
  final String nama;

  const Mahasiswa(this.nim, this.nama);
}

// Kumpulkan semua nama rute di satu tempat agar tidak salah ketik
class Rute {
  static const beranda = '/';
  static const detail = '/detail';
  static const tentang = '/tentang';
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.orange),
      initialRoute: Rute.beranda,
      onGenerateRoute: generateRoute, // dipanggil setiap pushNamed
    );
  }
}

// Satu fungsi pusat yang menerjemahkan NAMA rute menjadi HALAMAN
Route<dynamic> generateRoute(RouteSettings settings) {
  switch (settings.name) {
    case Rute.beranda:
      return MaterialPageRoute(builder: (_) => const BerandaScreen());

    case Rute.detail:
      // arguments bertipe Object?, jadi WAJIB dicek tipenya
      final args = settings.arguments;
      if (args is Mahasiswa) {
        return MaterialPageRoute(
          settings: settings,
          builder: (_) => DetailScreen(mahasiswa: args),
        );
      }
      return _halamanError('Argumen untuk ${settings.name} tidak valid');

    case Rute.tentang:
      return MaterialPageRoute(builder: (_) => const TentangScreen());

    default:
      return _halamanError('Rute "${settings.name}" tidak ditemukan');
  }
}

// Halaman cadangan agar aplikasi tidak crash ketika rute salah
MaterialPageRoute<dynamic> _halamanError(String pesan) {
  return MaterialPageRoute(
    builder: (_) => Scaffold(
      appBar: AppBar(title: const Text('Oops')),
      body: Center(child: Text(pesan)),
    ),
  );
}

class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  static const data = [
    Mahasiswa('H1D023001', 'Andi Pratama'),
    Mahasiswa('H1D023002', 'Bunga Lestari'),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
```

```
        actions: [
          IconButton(
            icon: const Icon(Icons.info_outline),
            onPressed: () => Navigator.pushNamed(context, Rute.tentang),
          ),
        ],
      ),
      body: ListView(
        children: [
          for (final m in data)
            ListTile(
              title: Text(m.nama),
              subtitle: Text(m.nim),
              // Kirim argumen lewat parameter "arguments"
              onTap: () => Navigator.pushNamed(context, Rute.detail, arguments: m),
            ),
          ListTile(
            title: const Text('Coba rute yang salah'),
            textColor: Colors.red,
            onTap: () => Navigator.pushNamed(context, '/tidak-ada'),
          ),
        ],
      ),
    );
  }
}

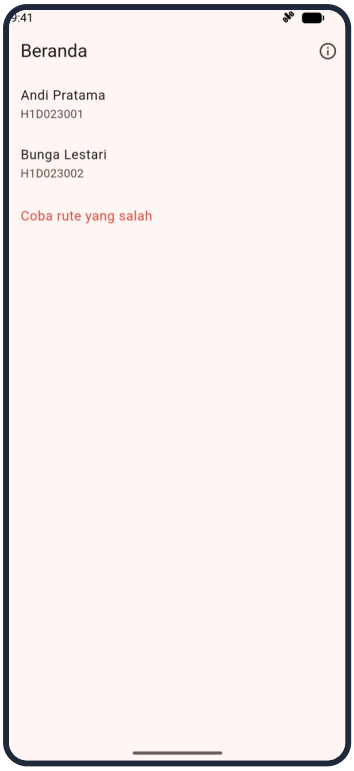
class DetailScreen extends StatelessWidget {
  final Mahasiswa mahasiswa;

  const DetailScreen({super.key, required this.mahasiswa});

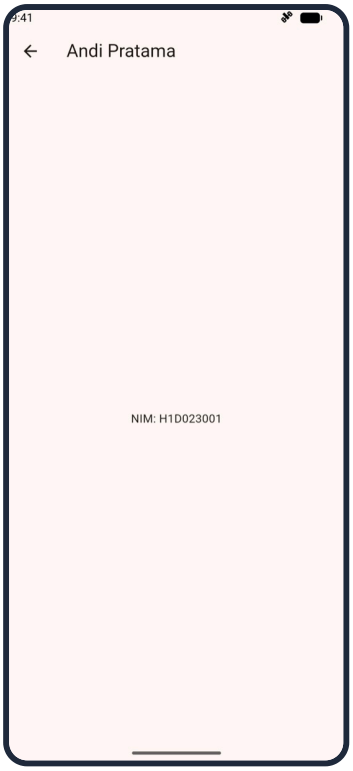
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(mahasiswa.nama)),
      body: Center(child: Text('NIM: ${mahasiswa.nim}')),
    );
  }
}

class TentangScreen extends StatelessWidget {
  const TentangScreen({super.key});

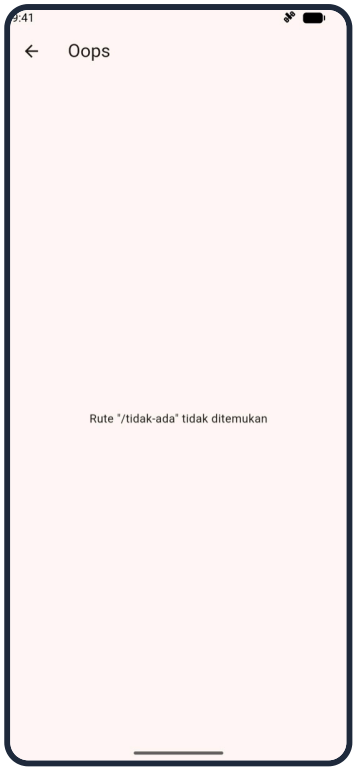
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tentang')),
      body: const Center(child: Text('Contoh Named Routes + onGenerateRoute')),
    );
  }
}
```



Beranda dengan named routes



Detail menerima argumen Mahasiswa



Rute tidak dikenal ditangani halaman error

Penjelasan:

- Class `Rute` berisi konstanta nama rute. Salah ketik `Rute.detial` akan langsung ditandai merah oleh editor, sedangkan salah ketik string `'/detial'` baru ketahuan saat aplikasi jalan.
- `Navigator.pushNamed(context, Rute.detail, arguments: m)` mengirim objek lewat `settings.arguments`.
- Di `generateRoute`, `if (args is Mahasiswa)` memeriksa tipe sekaligus melakukan *type promotion*, sehingga `args` bisa langsung dipakai sebagai `Mahasiswa`.
- Cabang `default` mengembalikan halaman error yang ramah, bukan crash.
- Varian lain: `pushReplacementNamed` dan `pushNamedAndRemoveUntil` bekerja sama seperti versi tanpa nama.

> 💡 **Named routes atau go_router?**
>
> Tim Flutter tidak lagi merekomendasikan named routes untuk aplikasi baru yang butuh *deep link* (membuka halaman tertentu dari URL/notifikasi). Untuk kebutuhan itu gunakan `go_router` (section 11). Namun named routes tetap penting dipahami karena banyak proyek lama memakainya.

⚠️ **Kesalahan umum**

- Melakukan cast paksa `settings.arguments as Mahasiswa` tanpa cek. Jika argumen lupa dikirim, aplikasi crash.
- Mengisi `home` sekaligus rute `'/'` di tabel `routes`. Flutter akan memberi error karena keduanya mendefinisikan halaman awal.
- Lupa meneruskan `settings` ke `MaterialPageRoute` sehingga `ModalRoute.of(context)?.settings.name` bernilai `null`.

7. NavigationBar Material 3 & IndexedStack

Aplikasi kampus biasanya punya 3–5 menu utama yang selalu terlihat di bawah layar. Di Material 3, komponennya adalah `NavigationBar` (pengganti `BottomNavigationBar` versi lama).

Konsep

- `NavigationBar` **bukan** Navigator. Berpindah tab tidak melakukan `push`; kita hanya mengganti widget yang ditampilkan di `body` berdasarkan indeks.
- Indeks tab aktif disimpan di `State` dan diubah dengan `setState` di `onDestinationSelected`.
- Setiap item adalah `NavigationDestination` dengan `icon`, `selectedIcon` (opsional), dan `label`.

Contoh lengkap 1: NavigationBar dasar

Struktur aplikasi standar kampus biasanya menggunakan menu tab bawah untuk berpindah antar fitur utama:

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const MainWrapperScreen(),
    );
  }
}

class MainWrapperScreen extends StatefulWidget {
  const MainWrapperScreen({super.key});

  @override
  State<MainWrapperScreen> createState() => _MainWrapperScreenState();
}

class _MainWrapperScreenState extends State<MainWrapperScreen> {
  int _currentIndex = 0;

  // Daftar layar untuk masing-masing tab (urutannya = urutan destinations)
  final List<Widget> _pages = const [
    HalamanTab(judul: "Beranda", ikon: Icons.home),
    HalamanTab(judul: "Jadwal Kuliah", ikon: Icons.calendar_month),
```

```
    HalamanTab(judul: "Kartu Rencana Studi", ikon: Icons.assignment),
    HalamanTab(judul: "Profil Mahasiswa", ikon: Icons.person),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Portal Akademik")),
      body: _pages[_currentIndex], // Menampilkan halaman sesuai tab aktif
      bottomNavigationBar: NavigationBar( // Komponen NavigationBar Material 3
        selectedIndex: _currentIndex,
        onDestinationSelected: (index) {
          setState(() => _currentIndex = index);
        },
        destinations: const [
          NavigationDestination(icon: Icon(Icons.home_outlined), selectedIcon: Icon(Icons.home),
label: "Beranda"),
          NavigationDestination(icon: Icon(Icons.calendar_month_outlined), selectedIcon:
Icon(Icons.calendar_month), label: "Jadwal"),
          NavigationDestination(icon: Icon(Icons.assignment_outlined), selectedIcon:
Icon(Icons.assignment), label: "KRS"),
          NavigationDestination(icon: Icon(Icons.person_outline), selectedIcon: Icon(Icons.person),
label: "Profil"),
        ],
      ),
    );
  }
}

// Halaman sederhana untuk tiap tab. Pada proyek nyata, ganti dengan
// BerandaScreen, JadwalScreen, dst. di file terpisah.
class HalamanTab extends StatelessWidget {
  final String judul;
  final IconData ikon;

  const HalamanTab({super.key, required this.judul, required this.ikon});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(ikon, size: 72, color: Colors.indigo),
          const SizedBox(height: 12),
          Text(judul, style: const TextStyle(fontSize: 22, fontWeight: FontWeight.bold)),
        ],
      ),
    );
  }
}
```



NavigationBar Material 3 dengan tab Jadwal aktif

Masalah: state tab hilang saat berpindah

Pada contoh di atas, `body: _pages[_currentIndex]` hanya memasang halaman aktif. Saat pindah tab, halaman lama **dibuang** dari pohon widget. Akibatnya isian form, posisi scroll, dan nilai di `State` tab tersebut hilang. Solusinya adalah `IndexedStack`:

```
// SEBELUM: hanya halaman aktif yang ada di pohon widget
// body: _pages[_currentIndex],

// SESUDAH: semua halaman tetap hidup, hanya satu yang terlihat
body: IndexedStack(
  index: _currentIndex,
  children: _pages,
),
```

`IndexedStack` membangun *semua* anaknya sekali, lalu hanya menampilkan anak pada posisi `index`. Anak yang tersembunyi tetap hidup beserta State-nya.

Contoh lengkap 2: IndexedStack menjaga state tab

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.green),
      home: const MainScreen(),
    );
  }
}

class MainScreen extends StatefulWidget {
  const MainScreen({super.key});

  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _index = 0;

  static const _judul = ['Presensi', 'Pengumuman', 'Akun'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_judul[_index])),
      // IndexedStack menjaga state setiap tab:
      // angka presensi & posisi scroll tidak hilang saat pindah tab
      body: IndexedStack(
        index: _index,
        children: const [
          PresensiTab(),
          PengumumanTab(),
          Center(child: Text('Halaman Akun')),
        ],
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _index,
        onDestinationSelected: (i) => setState(() => _index = i),
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.how_to_reg_outlined),
            selectedIcon: Icon(Icons.how_to_reg),
            label: 'Presensi',
          ),
          NavigationDestination(
            icon: Badge(label: Text('3'), child: Icon(Icons.campaign_outlined)),
            selectedIcon: Badge(label: Text('3'), child: Icon(Icons.campaign)),
            label: 'Pengumuman',
          ),
          NavigationDestination(
            icon: Icon(Icons.person_outline),
            selectedIcon: Icon(Icons.person),
            label: 'Akun',
          ),
        ],
```

```
      ),
    );
  }
}

// Tab 1: punya state lokal (jumlah kehadiran)
class PresensiTab extends StatefulWidget {
  const PresensiTab({super.key});

  @override
  State<PresensiTab> createState() => _PresensiTabState();
}

class _PresensiTabState extends State<PresensiTab> {
  int _hadir = 0;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text('Hadir: $_hadir pertemuan',
              style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: () => setState(() => _hadir++),
            child: const Text('Presensi Hari Ini'),
          ),
          const SizedBox(height: 8),
          const Text('Tambah angka, pindah tab, lalu kembali ke sini.'),
        ],
      ),
    );
  }
}

// Tab 2: list panjang (posisi scroll ikut tersimpan)
class PengumumanTab extends StatelessWidget {
  const PengumumanTab({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: 50,
      itemBuilder: (context, i) => ListTile(
        leading: const Icon(Icons.campaign),
        title: Text('Pengumuman ke-${i + 1}'),
      ),
    );
  }
}
```



Tab Presensi setelah tombol ditekan 3 kali



Pindah ke tab Akun lalu kembali: angka tetap 3

Penjelasan:

- `PresensiTab` adalah `StatefulWidget` dengan variabel `_hadir`. Berkat `IndexedStack`, angkanya tidak kembali ke 0 setelah pindah tab.
- `PengumumanTab` berisi 50 item. Gulir ke bawah, pindah tab, lalu kembali: posisi scroll tetap.
- `Badge` menampilkan angka kecil di atas ikon, berguna untuk notifikasi belum dibaca.
- Judul `AppBar` diambil dari list `_judul` sesuai tab aktif.

Pendekatan	Kelebihan	Kekurangan
`_pages[index]`	Hemat memori, tab dibangun saat dibuka	State tab hilang setiap pindah
`IndexedStack`	State tab terjaga	Semua tab dibangun di awal (hati-hati jika tiap tab memanggil API berat)

⚠️ **Kesalahan umum**

- Jumlah `destinations` kurang dari 2. `NavigationBar` butuh minimal dua item.
- Membuat list halaman di dalam `build()` tanpa `const`, lalu heran halaman ter-rebuild. Gunakan `const` jika halaman tidak butuh parameter.
- Memakai `Navigator.push` untuk berpindah tab. Tab bukan tumpukan; cukup ubah indeks.
- Masih memakai `BottomNavigationBar` lama di proyek baru. Untuk tampilan Material 3, pakai `NavigationBar`.

8. Menu Samping dengan Drawer

Drawer adalah panel menu yang muncul dari sisi kiri layar. Cocok untuk menu sekunder yang jumlahnya banyak (Jadwal, Transkrip, Pengaturan, Bantuan) dan tidak muat di `NavigationBar`.

Konsep

- Pasang drawer di parameter `drawer:` milik `Scaffold`. `AppBar` otomatis menampilkan ikon hamburger (≡).
- Drawer yang terbuka sebenarnya juga sebuah route di tumpukan. Karena itu, menutupnya cukup dengan `Navigator.pop(context)`.
- Saat memilih menu yang membuka halaman baru: **tutup drawer dulu**, baru `push`. Jika tidak, saat pengguna kembali, drawer masih terbuka.

Contoh lengkap: Drawer dengan header akun

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DashboardScreen(),
    );
  }
}

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Ikon hamburger (≡) muncul otomatis di AppBar karena ada drawer
      appBar: AppBar(title: const Text('Dashboard')),
      drawer: const MenuDrawer(),
      body: const Center(child: Text('Geser dari kiri atau tekan ikon ≡')),
    );
  }
}
```

```dart
class MenuDrawer extends StatelessWidget {
  const MenuDrawer({super.key});

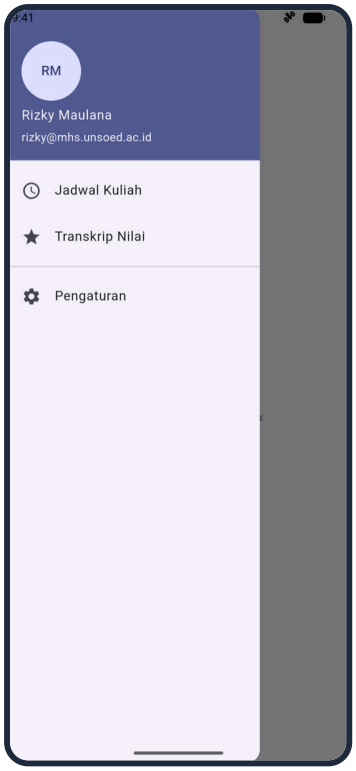
  // Tutup drawer DULU, baru buka halaman baru
  void _buka(BuildContext context, Widget halaman) {
    Navigator.pop(context); // menutup drawer (drawer juga sebuah rute!)
    Navigator.push(context, MaterialPageRoute(builder: (_) => halaman));
  }

  @override
  Widget build(BuildContext context) {
    return Drawer(
      child: ListView(
        padding: EdgeInsets.zero,
        children: [
          const UserAccountsDrawerHeader(
            accountName: Text('Rizky Maulana'),
            accountEmail: Text('rizky@mhs.unsoed.ac.id'),
            currentAccountPicture: CircleAvatar(child: Text('RM')),
          ),
          ListTile(
            leading: const Icon(Icons.schedule),
            title: const Text('Jadwal Kuliah'),
            onTap: () => _buka(context, const HalamanMenu(judul: 'Jadwal Kuliah')),
          ),
          ListTile(
            leading: const Icon(Icons.grade),
            title: const Text('Transkrip Nilai'),
            onTap: () => _buka(context, const HalamanMenu(judul: 'Transkrip Nilai')),
          ),
          const Divider(),
          ListTile(
            leading: const Icon(Icons.settings),
            title: const Text('Pengaturan'),
            onTap: () => _buka(context, const HalamanMenu(judul: 'Pengaturan')),
          ),
        ],
      ),
    );
  }
}

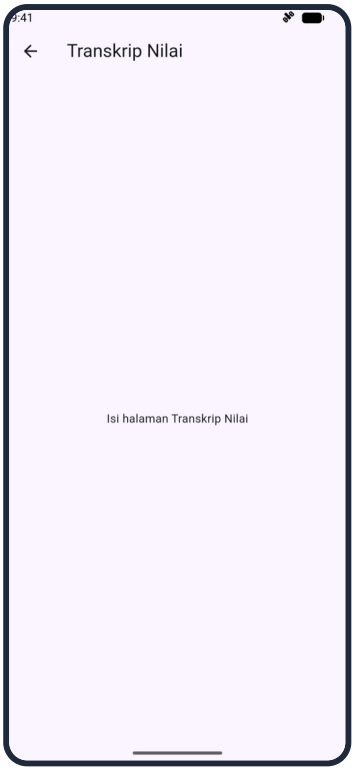
class HalamanMenu extends StatelessWidget {
  final String judul;

  const HalamanMenu({super.key, required this.judul});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(judul)),
      body: Center(child: Text('Isi halaman $judul')),
    );
  }
}
```



Drawer terbuka dengan header akun



Drawer ditutup lalu halaman baru di-push

Penjelasan:

- `UserAccountsDrawerHeader` adalah header siap pakai dengan nama, email, dan foto profil.
- `ListView` dengan `padding: EdgeInsets.zero` agar header menempel sampai atas layar.
- Fungsi `_buka` memanggil `pop` (menutup drawer) lalu `push` halaman tujuan.

Alternatif Material 3: NavigationDrawer

Jika menu drawer dipakai untuk berganti "halaman utama" (seperti tab), gunakan `NavigationDrawer` yang punya penanda menu aktif bawaan:

```
// Alternatif Material 3: NavigationDrawer
drawer: NavigationDrawer(
  selectedIndex: _menu,
  onDestinationSelected: (i) {
    setState(() => _menu = i);
    Navigator.pop(context); // tutup drawer
  },
  children: const [
    Padding(
      padding: EdgeInsets.fromLTRB(28, 16, 16, 10),
      child: Text('Portal Akademik'),
    ),
    NavigationDrawerDestination(icon: Icon(Icons.home), label: Text('Beranda')),
    NavigationDrawerDestination(icon: Icon(Icons.grade), label: Text('Nilai')),
  ],
),
```

💡 **Kapan pakai apa?**

`NavigationBar` untuk 3–5 menu utama yang sering dipakai. `Drawer` untuk menu tambahan. Keduanya boleh dipakai bersamaan dalam satu `Scaffold`.

⚠️ **Kesalahan umum**

- Menulis `Scaffold.of(context).openDrawer()` dengan `context` milik widget yang membuat `Scaffold` itu sendiri. Context tersebut berada di atas `Scaffold`, sehingga error. Bungkus tombol dengan `Builder` atau pisahkan menjadi widget sendiri.
- Lupa menutup drawer sebelum `push`.

9. TabBar & TabBarView

`TabBar` menampilkan deretan tab di bawah `AppBar`, dan `TabBarView` menampilkan isi tab yang bisa digeser kiri-kanan. Cocok untuk mengelompokkan data sejenis, misalnya jadwal per hari atau nilai per semester.

Tiga komponen yang harus selaras

1. `DefaultTabController(length: n)` : pengatur tab aktif.
2. `TabBar(tabs: [...])` : deretan judul tab, biasanya di `AppBar.bottom`.
3. `TabBarView(children: [...])` : isi setiap tab.

Contoh kecil

```
return DefaultTabController(
  length: 2, // HARUS sama dengan jumlah Tab dan jumlah anak TabBarView
  child: Scaffold(
    appBar: AppBar(
      title: const Text('Nilai'),
      bottom: const TabBar(
        tabs: [Tab(text: 'Semester Ganjil'), Tab(text: 'Semester Genap')],
      ),
    ),
    body: const TabBarView(
      children: [
        Center(child: Text('Nilai ganjil')),
        Center(child: Text('Nilai genap')),
      ],
    ),
  ),
);
```

Contoh lengkap: jadwal kuliah per hari

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.pink),
      home: const JadwalScreen(),
    );
```

```
  }
}

class Jadwal {
  final String jam;
  final String matkul;
  final String ruang;

  const Jadwal(this.jam, this.matkul, this.ruang);
}

const jadwalPerHari = {
  'Senin': [
    Jadwal('07.30', 'Pemrograman Mobile', 'Lab 1'),
    Jadwal('10.00', 'Basis Data', 'R. 201'),
  ],
  'Selasa': [Jadwal('08.00', 'Jaringan Komputer', 'Lab 3')],
  'Rabu': <Jadwal>[],
  'Kamis': [Jadwal('13.00', 'Kecerdasan Buatan', 'R. 305')],
  'Jumat': [Jadwal('07.30', 'Etika Profesi', 'Aula')],
};

class JadwalScreen extends StatelessWidget {
  const JadwalScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final hari = jadwalPerHari.keys.toList();

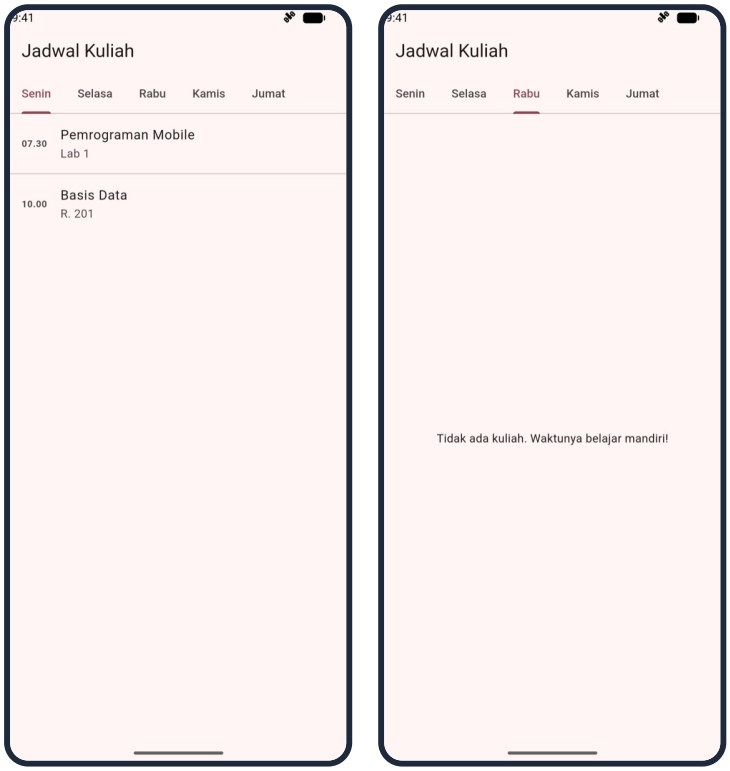
    return DefaultTabController(
      length: hari.length,
      child: Scaffold(
        appBar: AppBar(
          title: const Text('Jadwal Kuliah'),
          bottom: TabBar(
            isScrollable: true, // tab bisa digeser jika banyak
            tabAlignment: TabAlignment.start,
            tabs: [for (final h in hari) Tab(text: h)],
          ),
        ),
        body: TabBarView(
          children: [
            for (final h in hari) DaftarJadwal(items: jadwalPerHari[h]!),
          ],
        ),
      ),
    );
  }
}

class DaftarJadwal extends StatelessWidget {
  final List<Jadwal> items;

  const DaftarJadwal({super.key, required this.items});

  @override
  Widget build(BuildContext context) {
    if (items.isEmpty) {
      return const Center(child: Text('Tidak ada kuliah. Waktunya belajar mandiri!'));
```

```
    }
    return ListView.separated(
      itemCount: items.length,
      separatorBuilder: (context, i) => const Divider(height: 1),
      itemBuilder: (context, i) {
        final j = items[i];
        return ListTile(
          leading: Text(j.jam, style: const TextStyle(fontWeight: FontWeight.bold)),
          title: Text(j.matkul),
          subtitle: Text(j.ruang),
        );
      },
    );
  }
}
```



Tab Senin | Tab Rabu tanpa jadwal

Penjelasan:

- Daftar hari diambil dari `jadwalPerHari.keys`, sehingga `length`, jumlah `Tab`, dan jumlah anak `TabBarView` selalu sama.
- `isScrollable: true` + `tabAlignment: TabAlignment.start` membuat tab bisa digulir dan rata kiri saat jumlahnya banyak.
- Widget `DaftarJadwal` dipakai ulang untuk setiap hari, dengan tampilan khusus jika list kosong.

⚠️ **Kesalahan umum**

- `length` tidak sama dengan jumlah tab atau isi, menghasilkan error *"Controller's length property does not match"*.
- Lupa membungkus dengan `DefaultTabController`, menghasilkan error *"No TabController for TabBar"*.
- Memakai `TabBarView` di dalam `Column` tanpa `Expanded`, sehingga tingginya tidak terbatas.

10. Dialog & Bottom Sheet sebagai Route

Dialog, bottom sheet, dan date picker ternyata juga **route**. `showDialog` melakukan `push` sebuah `DialogRoute` ke tumpukan Navigator. Karena itu dialog ditutup dengan `Navigator.pop`, dan bisa mengembalikan nilai persis seperti halaman biasa.

Contoh kecil: dialog konfirmasi yang mengembalikan bool

```dart
// showDialog juga melakukan PUSH sebuah rute (DialogRoute)
final bool? yakin = await showDialog<bool>(
  context: context,
  builder: (dialogContext) => AlertDialog(
    title: const Text('Hapus data?'),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(dialogContext, false),
        child: const Text('Batal'),
      ),
      FilledButton(
        onPressed: () => Navigator.pop(dialogContext, true),
        child: const Text('Hapus'),
      ),
    ],
  ),
);
// yakin bisa true, false, atau null (ditutup dengan tap di luar dialog)
if (yakin == true) debugPrint('Data dihapus');
```

- `showDialog<bool>` mengembalikan `Future<bool?>`.
- Nilai `null` muncul ketika pengguna menutup dialog dengan tap di luar dialog atau tombol back.
- Builder dialog memberi `dialogContext` sendiri. Gunakan context ini untuk `pop` agar yang tertutup pasti dialognya.

Contoh lengkap: AlertDialog, bottom sheet, dan date picker

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

```
      theme: ThemeData(colorSchemeSeed: Colors.cyan),
      home: const DialogDemoScreen(),
    );
  }
}

class DialogDemoScreen extends StatefulWidget {
  const DialogDemoScreen({super.key});

  @override
  State<DialogDemoScreen> createState() => _DialogDemoScreenState();
}

class _DialogDemoScreenState extends State<DialogDemoScreen> {
  final List<String> _tugas = ['Laporan Praktikum 1', 'Kuis Basis Data', 'UTS Mobile'];
  String _prioritas = 'Sedang';
  DateTime? _deadline;

  // 1) AlertDialog yang mengembalikan bool
  Future<void> _hapus(int index) async {
    final yakin = await showDialog<bool>(
      context: context,
      barrierDismissible: false, // wajib pilih tombol, tap di luar tidak menutup
      builder: (dialogContext) => AlertDialog(
        icon: const Icon(Icons.delete_forever),
        title: const Text('Hapus tugas?'),
        content: Text('"${_tugas[index]}" akan dihapus permanen.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;
    setState(() => _tugas.removeAt(index));
  }

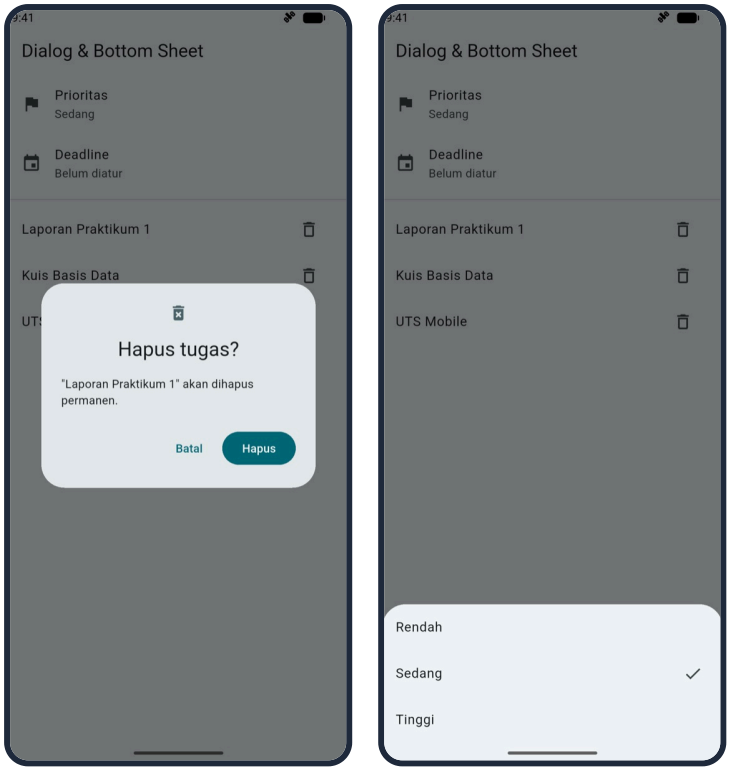
  // 2) Bottom sheet yang mengembalikan String
  Future<void> _pilihPrioritas() async {
    final pilihan = await showModalBottomSheet<String>(
      context: context,
      builder: (sheetContext) => SafeArea(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            for (final p in ['Rendah', 'Sedang', 'Tinggi'])
              ListTile(
                title: Text(p),
                trailing: p == _prioritas ? const Icon(Icons.check) : null,
                onTap: () => Navigator.pop(sheetContext, p),
              ),
          ],
        ),
```

```
      ),
    );
    if (pilihan == null || !mounted) return;
    setState(() => _prioritas = pilihan);
  }

  // 3) Date picker bawaan yang mengembalikan DateTime?
  Future<void> _pilihTanggal() async {
    final sekarang = DateTime.now();
    final tanggal = await showDatePicker(
      context: context,
      firstDate: sekarang,
      lastDate: DateTime(sekarang.year + 1),
      initialDate: _deadline ?? sekarang,
    );
    if (tanggal == null || !mounted) return;
    setState(() => _deadline = tanggal);
  }

  @override
  Widget build(BuildContext context) {
    final teksDeadline = _deadline == null
        ? 'Belum diatur'
        : '${_deadline!.day}/${_deadline!.month}/${_deadline!.year}';

    return Scaffold(
      appBar: AppBar(title: const Text('Dialog & Bottom Sheet')),
      body: ListView(
        children: [
          ListTile(
            leading: const Icon(Icons.flag),
            title: const Text('Prioritas'),
            subtitle: Text(_prioritas),
            onTap: _pilihPrioritas,
          ),
          ListTile(
            leading: const Icon(Icons.event),
            title: const Text('Deadline'),
            subtitle: Text(teksDeadline),
            onTap: _pilihTanggal,
          ),
          const Divider(),
          for (var i = 0; i < _tugas.length; i++)
            ListTile(
              title: Text(_tugas[i]),
              trailing: IconButton(
                tooltip: 'Hapus tugas',
                icon: const Icon(Icons.delete_outline),
                onPressed: () => _hapus(i),
              ),
            ),
        ],
      ),
    );
  }
}
```



AlertDialog konfirmasi hapus Bottom sheet pilihan prioritas

Penjelasan:

Fungsi	Tipe hasil	Ditutup dengan
`showDialog<bool>`	`bool?`	`Navigator.pop(dialogContext, true)`
`showModalBottomSheet<String>`	`String?`	`Navigator.pop(sheetContext, p)`
`showDatePicker`	`DateTime?`	Otomatis oleh tombol OK/Cancel

- `barrierDismissible: false` memaksa pengguna memilih tombol.
- Setiap fungsi memeriksa `!mounted` setelah `await` sebelum memanggil `setState`.
- `yakin != true` menangani dua kasus sekaligus: `false` dan `null`.

⚠️ **Kesalahan umum**

- Menulis `if (yakin)` padahal tipenya `bool?`. Dart menolak karena bisa `null`. Tulis `if (yakin == true)`.
- Memanggil `Navigator.pop(context)` dua kali (sekali untuk dialog, sekali "untuk jaga-jaga"), sehingga halaman di bawahnya ikut tertutup.
- Menampilkan SnackBar dengan `dialogContext` setelah dialog tertutup. Gunakan context halaman.

11. Pengenalan go_router

`go_router` adalah package routing resmi dari tim Flutter. Halaman diidentifikasi dengan **URL** (misalnya `/matkul/IF201`), sehingga mendukung deep link, tombol back browser di Flutter Web, dan *auth guard* terpusat.

Instalasi

```
flutter pub add go_router
```

Contoh kecil: rute dan path parameter

```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

// 1) Definisikan semua rute di SATU objek GoRouter
final GoRouter router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const BerandaScreen(),
    ),
    GoRoute(
      path: '/mahasiswa/:nim', // :nim = path parameter
      builder: (context, state) {
        final nim = state.pathParameters['nim']!;
        return ProfilScreen(nim: nim);
      },
    ),
  ],
);

// 2) Pakai MaterialApp.router, bukan MaterialApp biasa
void main() => runApp(MaterialApp.router(routerConfig: router));


// 3) Pindah halaman dengan URL
onPressed: () => context.push('/mahasiswa/H1D023001'),
```

- Semua rute didaftarkan sekali di `GoRouter(routes: [...])`.
- `:nim` pada path adalah **path parameter**. Nilainya dibaca dengan `state.pathParameters['nim']`. Nilainya selalu `String`.
- `MaterialApp.router(routerConfig: router)` menggantikan `MaterialApp(home: ...)`.

context.go vs context.push

Metode	Perilaku	Cocok untuk
`context.go('/x')`	Pindah ke lokasi `/x`. Tumpukan disusun ulang mengikuti struktur rute (sub-rute berada di atas induknya).	Pindah menu utama, setelah login/logout
`context.push('/x')`	Menumpuk `/x` di atas halaman saat ini, seperti `Navigator.push`.	Membuka detail, lalu kembali
`context.pop()`	Kembali satu halaman (bisa membawa hasil).	Tombol kembali/batal

Redirect sebagai auth guard

Fungsi `redirect` dipanggil sebelum setiap perpindahan halaman. Kembalikan path lain untuk membelokkan pengguna, atau `null` untuk membiarkan. Dengan `refreshListenable`, redirect juga dievaluasi ulang setiap kali status login berubah.

Contoh lengkap: login, sub-rute, path & query parameter

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah menjalankan: flutter pub add go_router
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

void main() {
  runApp(const MyApp());
}

// Penanda status login sederhana. ChangeNotifier adalah class bawaan Flutter;
// di sini cukup tahu bahwa notifyListeners() memberi kabar ke GoRouter.
class AuthState extends ChangeNotifier {
  bool _sudahLogin = false;
  bool get sudahLogin => _sudahLogin;

  void login() {
    _sudahLogin = true;
    notifyListeners();
  }

  void logout() {
    _sudahLogin = false;
    notifyListeners();
  }
}

final authState = AuthState();

final GoRouter router = GoRouter(
```

```
  initialLocation: '/',
  // Router mengevaluasi ulang redirect setiap kali authState berubah
  refreshListenable: authState,
  // AUTH GUARD: dipanggil sebelum setiap perpindahan halaman
  redirect: (context, state) {
    final sedangDiLogin = state.matchedLocation == '/login';
    if (!authState.sudahLogin && !sedangDiLogin) return '/login'; // belum login
    if (authState.sudahLogin && sedangDiLogin) return '/'; // sudah login
    return null; // null = boleh lanjut ke tujuan semula
  },
  routes: [
    GoRoute(
      path: '/login',
      builder: (context, state) => const LoginScreen(),
    ),
    GoRoute(
      path: '/',
      builder: (context, state) => const BerandaScreen(),
      routes: [
        // Sub-rute: URL lengkapnya menjadi /matkul/IF201
        GoRoute(
          path: 'matkul/:kode',
          builder: (context, state) {
            final kode = state.pathParameters['kode']!;
            // Query parameter opsional: /matkul/IF201?tab=nilai
            final tab = state.uri.queryParameters['tab'] ?? 'info';
            return DetailMatkulScreen(kode: kode, tab: tab);
          },
        ),
        GoRoute(
          path: 'tentang',
          builder: (context, state) => const TentangScreen(),
        ),
      ],
    ),
  ],
  // Halaman jika URL tidak dikenal
  errorBuilder: (context, state) => Scaffold(
    body: Center(child: Text('Halaman ${state.uri} tidak ditemukan')),
  ),
);

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      routerConfig: router,
    );
  }
}

class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Center(
        child: FilledButton(
          // Tidak perlu context.go('/'): redirect otomatis memindahkan
          // pengguna ke '/' setelah authState berubah.
          onPressed: authState.login,
          child: const Text('Masuk'),
        ),
      ),
    );
  }
}

class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  static const kodeMatkul = ['IF201', 'IF202', 'IF203'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
        actions: [
          IconButton(
            icon: const Icon(Icons.logout),
            onPressed: authState.logout, // redirect akan membawa ke /login
          ),
        ],
      ),
      body: ListView(
        children: [
          for (final kode in kodeMatkul)
            ListTile(
              title: Text('Mata kuliah $kode'),
              // push: tumpuk di atas Beranda (ada tombol back)
              onTap: () => context.push('/matkul/$kode'),
            ),
          ListTile(
            title: const Text('Buka nilai IF201 (query parameter)'),
            onTap: () => context.push('/matkul/IF201?tab=nilai'),
          ),
          ListTile(
            title: const Text('Tentang (pakai context.go)'),
            // go: pindah ke lokasi /tentang; tumpukan disusun ulang
            // sesuai struktur rute -> [Beranda, Tentang]
            onTap: () => context.go('/tentang'),
          ),
        ],
      ),
    );
  }
}

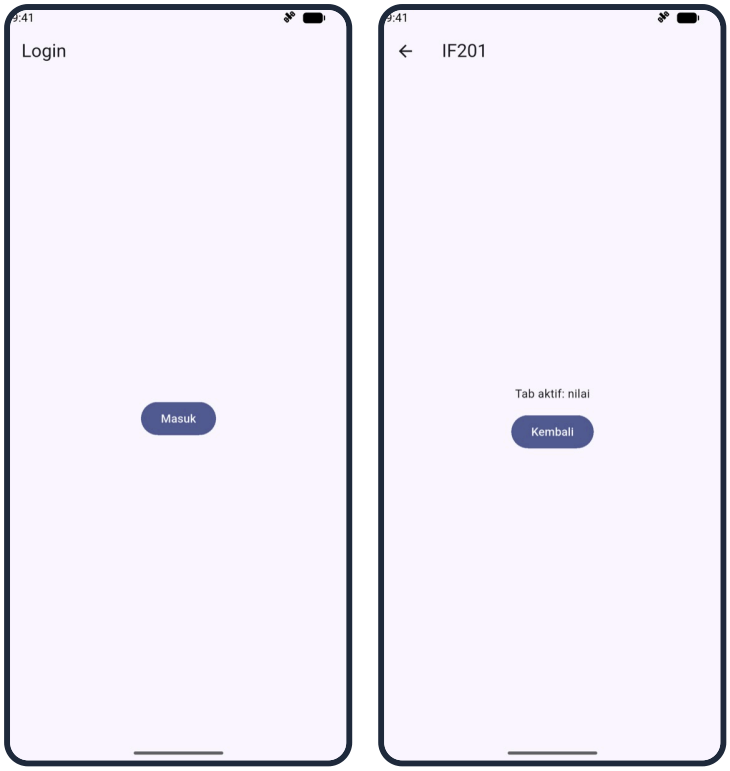
class DetailMatkulScreen extends StatelessWidget {
  final String kode;
  final String tab;
```

```
  const DetailMatkulScreen({super.key, required this.kode, required this.tab});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(kode)),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Tab aktif: $tab'),
            const SizedBox(height: 12),
            FilledButton(
              onPressed: () => context.pop(), // sama seperti Navigator.pop
              child: const Text('Kembali'),
            ),
          ],
        ),
      ),
    );
  }
}

class TentangScreen extends StatelessWidget {
  const TentangScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tentang')),
      body: const Center(child: Text('Contoh go_router')),
    );
  }
}
```



Belum login: redirect ke /login

Path parameter dan query parameter terbaca

Penjelasan:

- Saat aplikasi dibuka, `initialLocation` adalah `/`, tetapi `redirect` melihat pengguna belum login dan membelokkan ke `/login`.
- Tombol Masuk hanya mengubah `authState`. Karena `refreshListenable: authState`, router menjalankan redirect lagi dan memindahkan pengguna ke `/`. Logout bekerja dengan cara yang sama.
- `matkul/:kode` ditulis **tanpa garis miring awal** karena merupakan sub-rute dari `/`. URL lengkapnya `/matkul/IF201`.
- `state.uri.queryParameters['tab']` membaca bagian `?tab=nilai`. Query parameter cocok untuk data opsional.
- `errorBuilder` menampilkan halaman ketika URL tidak cocok dengan rute mana pun.

> ℹ️ **Hubungan dengan Modul 08 dan 13–14**
>
> Pada contoh ini `authState` adalah variabel global agar contoh tetap singkat. Setelah mempelajari GetX di Modul 08, status login akan disimpan di `AuthController` (sebuah `GetxController`). Proyek CRUD + Auth memakai pola yang setara: sebuah auth gate yang menampilkan Login atau Home berdasarkan sesi.

⚠️ **Kesalahan umum**

- Menulis sub-rute dengan garis miring awal (`'/matkul/:kode'` di dalam `routes` milik rute lain). go_router akan menolak konfigurasi tersebut.
- Redirect yang tidak pernah mengembalikan `null`, misalnya selalu `return '/login'`, menyebabkan *redirect loop*.
- Mencampur `MaterialApp(home: ...)` dengan `routerConfig`. Gunakan `MaterialApp.router`.
- Membuat objek `GoRouter` di dalam `build()`. Setiap rebuild akan membuat router baru dan posisi halaman hilang. Buat sekali saja (variabel top-level atau di State).

12. PopScope: Konfirmasi Sebelum Kembali

Kadang tombol back perlu "dicegat", misalnya ketika form sudah diisi tetapi belum disimpan. Widget yang dipakai adalah `PopScope` (pengganti `WillPopScope` yang sudah dihapus dari Flutter).

Dua parameter penting

- `canPop` : `true` berarti back boleh langsung menutup halaman. `false` berarti back diblokir.
- `onPopInvokedWithResult: (didPop, result)` : dipanggil setiap kali ada upaya kembali. `didPop` bernilai `true` jika halaman sudah tertutup, `false` jika diblokir.

Contoh kecil

```
return PopScope(
  canPop: false, // blokir tombol/gestur back
  onPopInvokedWithResult: (bool didPop, Object? result) {
    if (didPop) return; // sudah keluar, tidak perlu apa-apa
    // Di sini: tampilkan dialog konfirmasi, simpan draf, dll.
    debugPrint('Back ditekan tapi diblokir');
  },
  child: const Scaffold(body: Center(child: Text('Form'))),
);
```

Contoh lengkap: konfirmasi sebelum membuang isian form

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.brown),
      home: const BerandaScreen(),
    );
  }
}

class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: const Text('Beranda')),
      body: Center(
        child: FilledButton(
          onPressed: () => Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const FormIzinScreen()),
          ),
          child: const Text('Ajukan Izin Kuliah'),
        ),
      ),
    );
  }
}

class FormIzinScreen extends StatefulWidget {
  const FormIzinScreen({super.key});

  @override
  State<FormIzinScreen> createState() => _FormIzinScreenState();
}

class _FormIzinScreenState extends State<FormIzinScreen> {
  final _alasanController = TextEditingController();
  bool _adaPerubahan = false;

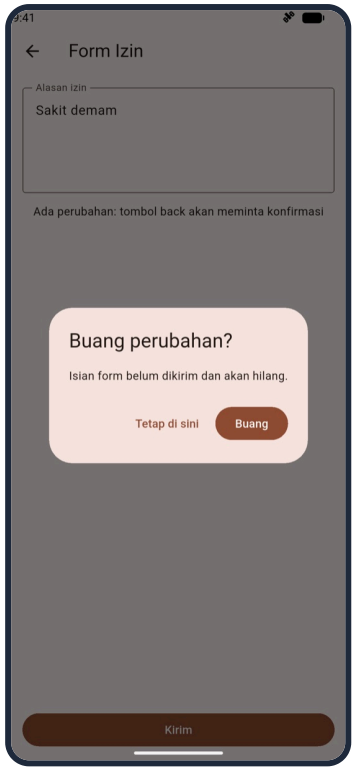
  @override
  void dispose() {
    _alasanController.dispose();
    super.dispose();
  }

  Future<bool> _konfirmasiKeluar() async {
    final keluar = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Buang perubahan?'),
        content: const Text('Isian form belum dikirim dan akan hilang.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Tetap di sini'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Buang'),
          ),
        ],
      ),
    );
    return keluar ?? false;
  }

  void _kirim() {
    // Setelah terkirim, form boleh ditutup tanpa konfirmasi
    setState(() => _adaPerubahan = false);
    Navigator.pop(context);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Pengajuan izin terkirim')),
```

```
    );
  }

  @override
  Widget build(BuildContext context) {
    return PopScope(
      // Jika tidak ada perubahan -> boleh langsung kembali
      canPop: !_adaPerubahan,
      onPopInvokedWithResult: (didPop, result) async {
        if (didPop) return;
        final keluar = await _konfirmasiKeluar();
        // Pakai context milik State, cek mounted setelah await
        if (keluar && context.mounted) {
          Navigator.pop(context);
        }
      },
      child: Scaffold(
        appBar: AppBar(title: const Text('Form Izin')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _alasanController,
                maxLines: 4,
                decoration: const InputDecoration(
                  labelText: 'Alasan izin',
                  border: OutlineInputBorder(),
                ),
                onChanged: (teks) {
                  final berubah = teks.trim().isNotEmpty;
                  if (berubah != _adaPerubahan) {
                    setState(() => _adaPerubahan = berubah);
                  }
                },
              ),
              const SizedBox(height: 12),
              Text(_adaPerubahan
                  ? 'Ada perubahan: tombol back akan meminta konfirmasi'
                  : 'Belum ada perubahan: tombol back langsung kembali'),
              const Spacer(),
              SizedBox(
                width: double.infinity,
                child: FilledButton(onPressed: _kirim, child: const Text('Kirim')),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```



Tombol back memicu dialog konfirmasi

Penjelasan:

1. `canPop: !_adaPerubahan`: jika form masih kosong, back berjalan normal.
2. Begitu pengguna mengetik, `_adaPerubahan` menjadi `true`, sehingga `canPop` menjadi `false`.
3. Saat back ditekan, `onPopInvokedWithResult` dipanggil dengan `didPop == false`, lalu dialog konfirmasi ditampilkan.
4. Jika pengguna memilih "Buang", kita menutup halaman sendiri dengan `Navigator.pop(context)`. Pemanggilan `pop` secara langsung dari kode tetap berjalan walaupun `canPop` bernilai `false`.
5. Tombol panah di `AppBar`, tombol back Android, dan gestur back semuanya melewati `PopScope`.

> ⚠️ **Kesalahan umum**
>
> - Memakai `WillPopScope` atau parameter `onPopInvoked` dari tutorial lama. Keduanya sudah usang; gunakan `PopScope` + `onPopInvokedWithResult`.
> - Tidak mengecek `didPop`. Jika halaman sudah tertutup lalu kita memanggil `Navigator.pop` lagi, halaman di bawahnya ikut tertutup.
> - Membuat `canPop: false` permanen tanpa memberi jalan keluar. Pengguna akan "terjebak" di halaman itu.

13. Studi Kasus: Portal Akademik Multi-Halaman

Saatnya menggabungkan semua materi menjadi satu aplikasi **Portal Akademik** multi-halaman: login dengan `pushReplacement`, tab utama dengan `NavigationBar` + `IndexedStack`, detail mata kuliah yang menerima data lewat constructor dan mengembalikan hasil, dialog logout, `pushAndRemoveUntil`, serta `PopScope`.

Struktur folder

```
lib/
├── main.dart
├── models/
|   └── matkul.dart
├── data/
|   └── dummy_data.dart
└── screens/
    ├── login_screen.dart
    ├── main_screen.dart
    └── detail_matkul_screen.dart
```

Peta navigasi

```
LoginScreen --pushReplacement--> MainScreen (tab: Beranda | Mata Kuliah | Profil)
MainScreen  --push<bool>------> DetailMatkulScreen --pop(true)--> MainScreen
MainScreen  --showDialog<bool>--> konfirmasi --pushAndRemoveUntil--> LoginScreen
```

Langkah 1: model dan data

`lib/models/matkul.dart`

```dart
// lib/models/matkul.dart
class Matkul {
  final String kode;
  final String nama;
  final int sks;
  final String dosen;
  final String deskripsi;

  const Matkul({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.dosen,
    required this.deskripsi,
  });
}
```

`lib/data/dummy_data.dart`

```dart
// lib/data/dummy_data.dart
import '../models/matkul.dart';
```

```dart
const List<Matkul> semuaMatkul = [
  Matkul(
    kode: 'IF201',
    nama: 'Pemrograman Mobile',
    sks: 3,
    dosen: 'Dr. Rina Wulandari',
    deskripsi: 'Membangun aplikasi Android & iOS dengan Flutter.',
  ),
  Matkul(
    kode: 'IF202',
    nama: 'Basis Data',
    sks: 3,
    dosen: 'Budi Santoso, M.Kom.',
    deskripsi: 'Perancangan basis data relasional dan SQL.',
  ),
  Matkul(
    kode: 'IF203',
    nama: 'Jaringan Komputer',
    sks: 2,
    dosen: 'Sari Dewi, M.T.',
    deskripsi: 'Model OSI, TCP/IP, dan praktik konfigurasi jaringan.',
  ),
  Matkul(
    kode: 'IF204',
    nama: 'Kecerdasan Buatan',
    sks: 3,
    dosen: 'Dr. Agus Riyadi',
    deskripsi: 'Pencarian, logika, dan dasar machine learning.',
  ),
];
```

Langkah 2: titik masuk aplikasi

lib/main.dart

```dart
// lib/main.dart
import 'package:flutter/material.dart';

import 'screens/login_screen.dart';

void main() {
  runApp(const PortalAkademikApp());
}

class PortalAkademikApp extends StatelessWidget {
  const PortalAkademikApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Portal Akademik',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const LoginScreen(),
    );
  }
}
```

Langkah 3: halaman login

`lib/screens/login_screen.dart`

```dart
// lib/screens/login_screen.dart
import 'package:flutter/material.dart';

import 'main_screen.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _nimController = TextEditingController();

  @override
  void dispose() {
    _nimController.dispose();
    super.dispose();
  }

  void _login() {
    final nim = _nimController.text.trim();
    if (nim.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('NIM wajib diisi')),
      );
      return;
    }
    // Login sukses: ganti Login dengan MainScreen (tidak bisa back ke login)
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => MainScreen(nim: nim)),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Icon(Icons.school, size: 80),
              const SizedBox(height: 16),
              Text('Portal Akademik', style: Theme.of(context).textTheme.headlineMedium),
              const SizedBox(height: 24),
              TextField(
                controller: _nimController,
                decoration: const InputDecoration(
                  labelText: 'NIM',
                  border: OutlineInputBorder(),
```

```
              ),
            ),
            const SizedBox(height: 16),
            SizedBox(
              width: double.infinity,
              child: FilledButton(onPressed: _login, child: const Text('Masuk')),
            ),
          ],
        ),
      ),
    ),
  );
  }
}
```

Langkah 4: halaman utama dengan tab

lib/screens/main_screen.dart

```
// lib/screens/main_screen.dart
import 'package:flutter/material.dart';

import '../data/dummy_data.dart';
import '../models/matkul.dart';
import 'detail_matkul_screen.dart';
import 'login_screen.dart';

class MainScreen extends StatefulWidget {
  final String nim;

  const MainScreen({super.key, required this.nim});

  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _tab = 0;
  final List<Matkul> _diambil = [];

  int get _totalSks => _diambil.fold(0, (total, m) => total + m.sks);

  // Buka detail, tunggu jawaban "ambil" (true) atau tidak
  Future<void> _bukaDetail(Matkul matkul) async {
    final ambil = await Navigator.push<bool>(
      context,
      MaterialPageRoute(
        builder: (_) => DetailMatkulScreen(
          matkul: matkul,
          sudahDiambil: _diambil.contains(matkul),
        ),
      ),
    );
    if (ambil != true || !mounted) return;
    setState(() => _diambil.add(matkul));
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        backgroundColor: Colors.green,
```

```
        content: Text('${matkul.nama} masuk KRS'),
      ),
    );
  }

  Future<void> _logout() async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Keluar dari akun?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Logout'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;
    // Bersihkan SELURUH tumpukan, sisakan Login saja
    Navigator.pushAndRemoveUntil(
      context,
      MaterialPageRoute(builder: (_) => const LoginScreen()),
      (route) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    final tabs = [
      _BerandaTab(nim: widget.nim, totalSks: _totalSks),
      _MatkulTab(diambil: _diambil, onTap: _bukaDetail),
      _ProfilTab(nim: widget.nim, onLogout: _logout),
    ];

    return PopScope(
      // Jika bukan di tab Beranda, tombol back kembali ke Beranda dulu
      canPop: _tab == 0,
      onPopInvokedWithResult: (didPop, result) {
        if (!didPop) setState(() => _tab = 0);
      },
      child: Scaffold(
        appBar: AppBar(title: const Text('Portal Akademik')),
        body: IndexedStack(index: _tab, children: tabs),
        bottomNavigationBar: NavigationBar(
          selectedIndex: _tab,
          onDestinationSelected: (i) => setState(() => _tab = i),
          destinations: [
            const NavigationDestination(icon: Icon(Icons.home), label: 'Beranda'),
            NavigationDestination(
              icon: Badge(
                isLabelVisible: _diambil.isNotEmpty,
                label: Text('${_diambil.length}'),
                child: const Icon(Icons.book),
              ),
```

```
              label: 'Mata Kuliah',
            ),
            const NavigationDestination(icon: Icon(Icons.person), label: 'Profil'),
          ],
        ),
      ),
    );
  }
}

class _BerandaTab extends StatelessWidget {
  final String nim;
  final int totalSks;

  const _BerandaTab({required this.nim, required this.totalSks});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text('Selamat datang, $nim'),
          const SizedBox(height: 8),
          Text('Total SKS diambil: $totalSks',
              style: Theme.of(context).textTheme.headlineSmall),
        ],
      ),
    );
  }
}

class _MatkulTab extends StatelessWidget {
  final List<Matkul> diambil;
  final ValueChanged<Matkul> onTap;

  const _MatkulTab({required this.diambil, required this.onTap});

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: [
        for (final m in semuaMatkul)
          ListTile(
            title: Text(m.nama),
            subtitle: Text('${m.kode} • ${m.sks} SKS'),
            trailing: diambil.contains(m)
                ? const Icon(Icons.check_circle, color: Colors.green)
                : const Icon(Icons.chevron_right),
            onTap: () => onTap(m),
          ),
      ],
    );
  }
}

class _ProfilTab extends StatelessWidget {
  final String nim;
  final VoidCallback onLogout;
```

```
  const _ProfilTab({required this.nim, required this.onLogout});

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: [
        ListTile(leading: const Icon(Icons.badge), title: const Text('NIM'), subtitle: Text(nim)),
        ListTile(
          leading: const Icon(Icons.logout, color: Colors.red),
          title: const Text('Logout'),
          onTap: onLogout,
        ),
      ],
    );
  }
}
```

Langkah 5: halaman detail

lib/screens/detail_matkul_screen.dart

```
// lib/screens/detail_matkul_screen.dart
import 'package:flutter/material.dart';

import '../models/matkul.dart';

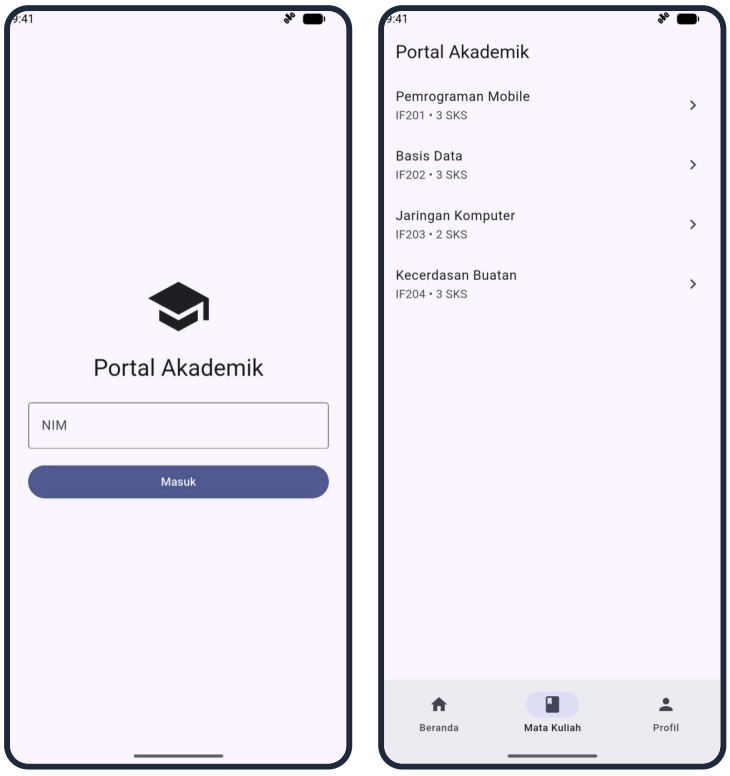
class DetailMatkulScreen extends StatelessWidget {
  final Matkul matkul;
  final bool sudahDiambil;

  const DetailMatkulScreen({
    super.key,
    required this.matkul,
    required this.sudahDiambil,
  });

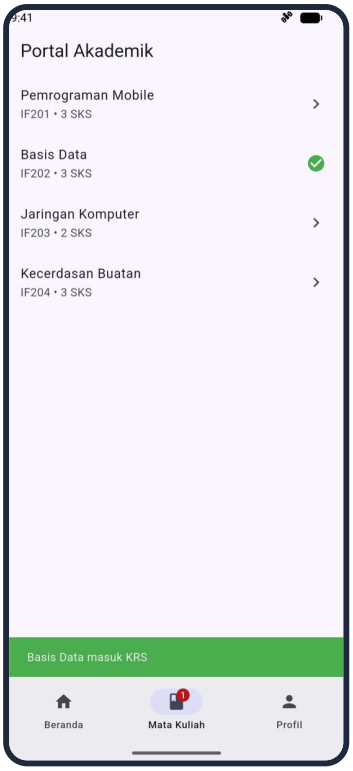
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(matkul.kode)),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(matkul.nama, style: Theme.of(context).textTheme.headlineSmall),
            const SizedBox(height: 8),
            Text('${matkul.sks} SKS • ${matkul.dosen}'),
            const SizedBox(height: 16),
            Text(matkul.deskripsi),
            const Spacer(),
            SizedBox(
              width: double.infinity,
              child: FilledButton.icon(
                icon: const Icon(Icons.add_task),
                label: Text(sudahDiambil ? 'Sudah di KRS' : 'Daftar Matkul Ini'),
                // null = tombol nonaktif
```

```
                onPressed: sudahDiambil ? null : () => Navigator.pop(context, true),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Hasil



Halaman login | Tab Mata Kuliah



Matkul masuk KRS: centang hijau dan badge

Penjelasan:

- **Login** memakai `pushReplacement`, jadi tombol back di MainScreen tidak kembali ke login. NIM dikirim ke `MainScreen` lewat constructor.
- **MainScreen** menyimpan daftar `_diambil` dan mengoper data serta callback ke tiap tab lewat constructor (`_MatkulTab(diambil: ..., onTap: ...)`).
- **Detail** mengembalikan `true` saat tombol "Daftar Matkul Ini" ditekan. MainScreen menunggu hasil itu dengan `await Navigator.push<bool>`.
- **PopScope** di MainScreen: jika pengguna sedang di tab selain Beranda, tombol back memindahkan ke Beranda dulu, bukan menutup aplikasi.
- **Logout** meminta konfirmasi lewat dialog, lalu membersihkan tumpukan dengan `pushAndRemoveUntil`.

> ℹ️ **Menuju Modul 08**
>
> Perhatikan bahwa `_diambil` harus dioper dari `MainScreen` ke tab-tabnya lewat constructor dan callback. Bayangkan jika halaman detail, halaman ringkasan KRS, dan badge di AppBar semuanya butuh data yang sama: constructor akan penuh dengan parameter "titipan". Masalah ini disebut **prop drilling**, dan diselesaikan dengan GetX (controller + pola MVC) di modul berikutnya.

💡 **Tantangan tambahan**

- Tambahkan tombol "Batalkan" di halaman detail jika mata kuliah sudah diambil, lalu kembalikan nilai `false` untuk menghapusnya dari `_diambil`.
- Tambahkan `Drawer` berisi menu Tentang Aplikasi.
- Ubah navigasi aplikasi ini menjadi `go_router` dengan rute `/login`, `/`, dan `/matkul/:kode`.

Tugas Praktikum: Praktikum 7: Portal Akademik dengan Navigasi Tab & Halaman Detail

Sasaran Praktikum

Merancang alur navigasi aplikasi kampus yang lengkap mulai dari Splash, Login, Main Tabs, hingga Halaman Detail Berargumen, dialog konfirmasi, dan pencegatan tombol back.

Langkah Pengerjaan

1. Buat proyek baru dan susun folder `lib/models`, `lib/data`, dan `lib/screens` seperti studi kasus section 13.
2. Buat `SplashScreen` yang setelah 2 detik berpindah ke `LoginScreen` dengan `Navigator.pushReplacement` (jangan lupa cek `mounted`).
3. Buat alur layar: `LoginScreen` → `MainNavigationScreen`.
4. Gunakan `Navigator.pushReplacement` dari Login agar mahasiswa tidak bisa menekan tombol back kembali ke layar login setelah login sukses.
5. Di dalam `MainNavigationScreen`, pasang `NavigationBar` Material 3 dengan 3 tab: (1) Beranda Informasi, (2) Daftar Mata Kuliah, (3) Profil Pengguna. Gunakan `IndexedStack` agar posisi scroll tab tidak hilang.
6. Pada tab Daftar Mata Kuliah, ketika salah satu item ditekan, pindah ke `DetailMataKuliahScreen` dengan membawa objek mata kuliah (nama dosen dan deskripsi silabus) lewat constructor.
7. Sediakan tombol 'Daftar Matkul Ini' pada Halaman Detail yang mengembalikan data hasil konfirmasi ke halaman sebelumnya (`await Navigator.push<bool>`) dan menampilkan notifikasi SnackBar hijau.
8. Tambahkan `Drawer` berisi menu Jadwal dan Tentang Aplikasi. Tutup drawer sebelum membuka halaman baru.
9. Halaman Jadwal memakai `TabBar` + `TabBarView` untuk hari Senin sampai Jumat.
10. Tambahkan tombol 'Logout' di tab Profil yang menampilkan `AlertDialog` konfirmasi, lalu mengarahkan kembali ke Login menggunakan `pushAndRemoveUntil`.
11. Pasang `PopScope` di MainNavigationScreen: jika tab aktif bukan Beranda, tombol back memindahkan ke tab Beranda terlebih dahulu.
12. Bonus: implementasikan ulang rute utama dengan `go_router` (rute `/login`, `/`, `/matkul/:kode`) dan `redirect` sebagai auth guard.

Output yang Diharapkan

Alur navigasi aplikasi yang rapi: splash tidak bisa dikembalikan, login tidak tersisa di stack, state tab terjaga, data timbal-balik berfungsi, logout membersihkan seluruh stack, dan tombol back berperilaku sesuai PopScope.

Rubrik Penilaian

Penggunaan push, pop, pushReplacement, dan pushAndRemoveUntil yang tepat (30%), Passing data via constructor dan return result (25%), NavigationBar + IndexedStack, Drawer, dan TabBar yang bersih (25%), Dialog konfirmasi dan PopScope (10%), Struktur folder dan kerapian kode (10%). Bonus go_router +10%.

Kuis Pemahaman

1. Mengapa setelah pengguna berhasil login, kita sebaiknya menggunakan 'Navigator.pushReplacement' alih-alih 'Navigator.push' biasa?

A. Agar layar Login terhapus dari navigation stack, sehingga jika pengguna menekan tombol back di perangkat, aplikasi keluar dan tidak kembali lagi ke layar formulir login

B. Karena push biasa hanya boleh digunakan untuk membuka website di browser

C. Agar warna AppBar otomatis berubah menjadi biru

D. Karena pushReplacement mengompres file gambar lebih cepat

2. Bagaimana cara menerima data balik (return value) yang dikirim oleh sebuah halaman saat halaman tersebut ditutup dengan 'Navigator.pop(context, data)'?

A. Menambahkan keyword 'await' di depan pemanggilan 'Navigator.push'

B. Menyimpan data di dalam file pubspec.yaml

C. Menggunakan fungsi setState() di dalam initState()

D. Mematikan emulator ponsel

3. Apa perbedaan utama context.go() dan context.push() pada go_router?

A. go menyusun ulang tumpukan sesuai struktur rute tujuan, sedangkan push menumpuk halaman baru di atas halaman saat ini

B. go hanya untuk Flutter Web, push hanya untuk Android

C. push menghapus semua halaman, go tidak

D. Keduanya identik, hanya beda nama

4. Pada Navigator.pushAndRemoveUntil(context, route, (route) ⇒ false), apa arti predicate '(route) ⇒ false'?

A. Hapus semua rute lama sehingga hanya rute baru yang tersisa di tumpukan

B. Jangan hapus rute apa pun

C. Hapus hanya rute paling atas

D. Batalkan navigasi

5. Mengapa body: IndexedStack(index: i, children: pages) dipakai alih-alih body: pages[i] pada NavigationBar?

A. Agar state setiap tab (isian, posisi scroll, nilai State) tetap terjaga saat berpindah tab

B. Agar aplikasi lebih hemat memori

C. Karena NavigationBar mewajibkan IndexedStack

D. Agar animasi pindah tab lebih cepat

6. Di dalam onGenerateRoute, bagaimana cara aman membaca settings.arguments yang seharusnya bertipe Mahasiswa?

A. Memeriksa dengan if (args is Mahasiswa) lalu mengembalikan halaman error jika tidak cocok

B. Langsung menulis settings.arguments as Mahasiswa

C. Mengubah arguments menjadi String dengan toString()

D. Menyimpan argumen di variabel global

7. Widget apa yang dipakai di Flutter terbaru untuk mencegat tombol back dan menampilkan konfirmasi keluar?

A. PopScope dengan canPop dan onPopInvokedWithResult

B. WillPopScope dengan onWillPop

C. BackButtonListener di main()

D. GestureDetector dengan onBack

8. showDialog<bool>(...) dipanggil dan pengguna menutup dialog dengan tap di luar kotak dialog. Nilai apa yang diterima?

A. null

B. false

C. true

D. Aplikasi crash

Kunci Jawaban & Pembahasan

1. A — pushReplacement mengganti rute saat ini di stack dengan rute baru. Dengan begitu, rute login tidak tersisa di stack riwayat dan pengguna tidak akan kembali ke login saat menekan tombol back.

2. A — Karena Navigator.push mengembalikan Future<T>, kita dapat menaruh keyword 'await' di depannya: final result = await Navigator.push(...). Ketika layar tujuan ditutup dengan Navigator.pop(context, data), variabel result akan berisi nilai tersebut.

3. A — context.go berpindah ke sebuah lokasi URL dan membangun tumpukan berdasarkan hierarki rute (sub-rute di atas induknya). context.push menumpuk rute di atas halaman saat ini, mirip Navigator.push, sehingga tombol back kembali ke halaman sebelumnya.

4. A — Predicate dipanggil untuk tiap rute lama. Rute dibuang selama predicate mengembalikan false. Karena selalu false, semua rute lama dibuang. Ini pola standar untuk logout.

5. A — IndexedStack membangun semua anak dan hanya menampilkan satu. Anak yang tersembunyi tidak dibuang, sehingga State-nya tetap hidup. Konsekuensinya semua tab dibangun di awal.

6. A — settings.arguments bertipe Object?. Pemeriksaan 'is' mencegah crash ketika argumen lupa dikirim atau tipenya salah, sekaligus melakukan type promotion.

7. A — WillPopScope sudah dihapus. PopScope memakai canPop untuk memblokir back, dan onPopInvokedWithResult(didPop, result) untuk bereaksi, misalnya menampilkan dialog lalu memanggil Navigator.pop sendiri.

8. A — Dialog adalah route. Jika ditutup tanpa Navigator.pop(context, nilai), Future selesai dengan null. Karena itu kondisi ditulis if (yakin == true), bukan if (yakin).